

Lab Tasks

Name: Muhammad Faisal

Roll no: 2022F-BCS-152

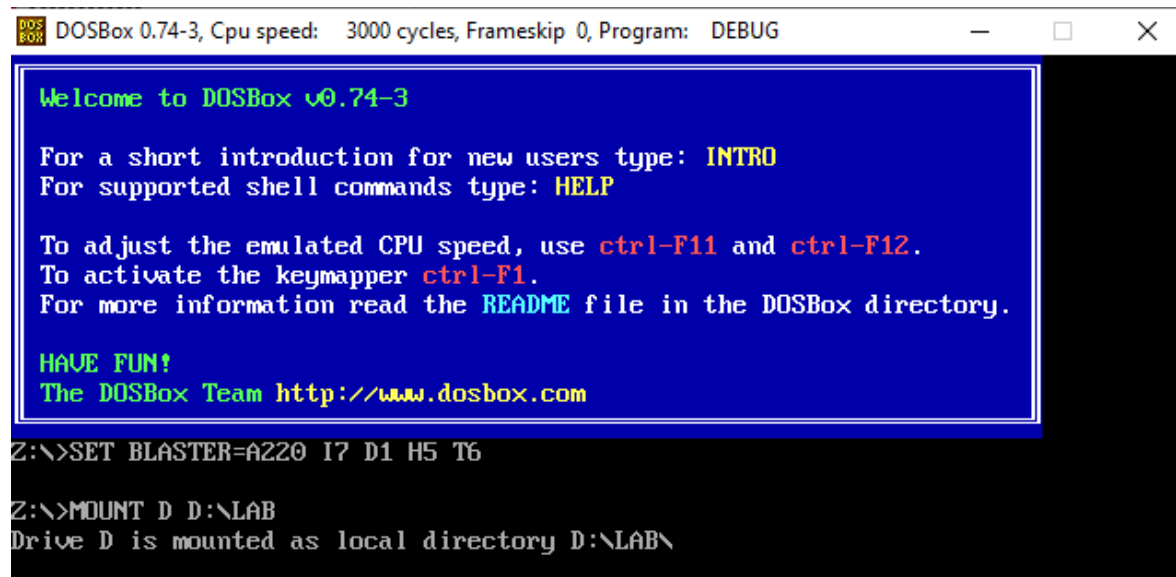
Section: C

Instructor Name: Sir Yazdani

Course: Computer Organization & Assembly Language

Program: (BS) - Computer Science

MOUNT



```
DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG

Welcome to DOSBox v0.74-3

For a short introduction for new users type: INTRO
For supported shell commands type: HELP

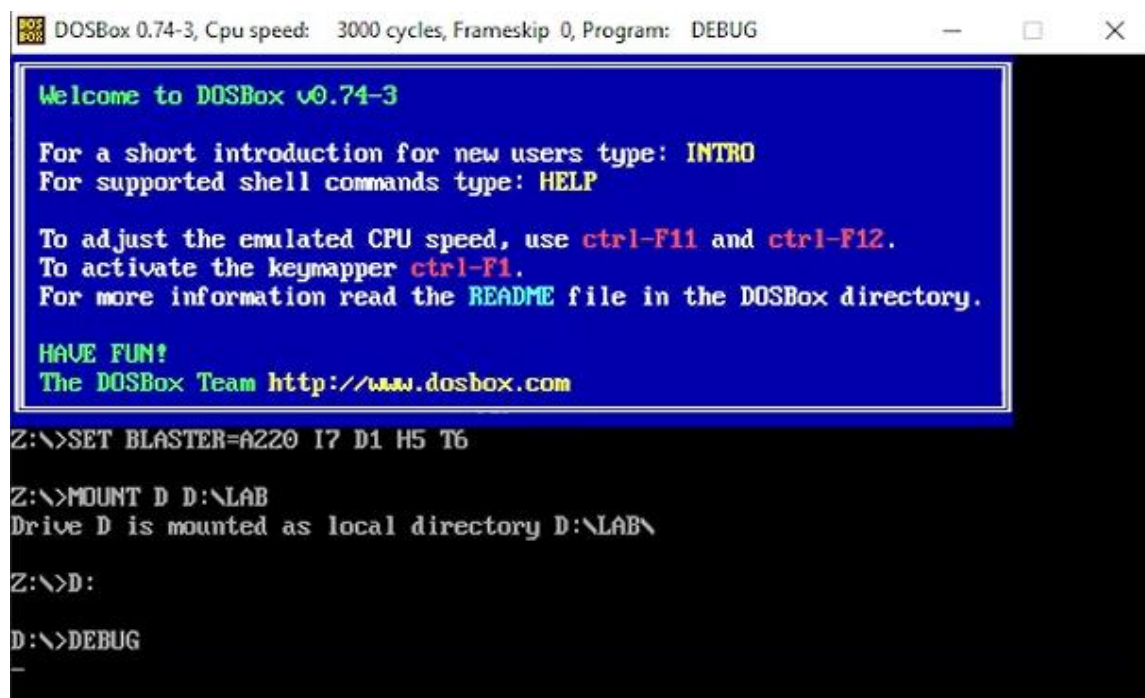
To adjust the emulated CPU speed, use ctrl-F11 and ctrl-F12.
To activate the keymapper ctrl-F1.
For more information read the README file in the DOSBox directory.

HAVE FUN!
The DOSBox Team http://www.dosbox.com

Z:\>SET BLASTER=A220 I7 D1 H5 T6

Z:\>MOUNT D D:\LAB
Drive D is mounted as local directory D:\LAB\
```

Entering DEBUG



```
DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG

Welcome to DOSBox v0.74-3

For a short introduction for new users type: INTRO
For supported shell commands type: HELP

To adjust the emulated CPU speed, use ctrl-F11 and ctrl-F12.
To activate the keymapper ctrl-F1.
For more information read the README file in the DOSBox directory.

HAVE FUN!
The DOSBox Team http://www.dosbox.com

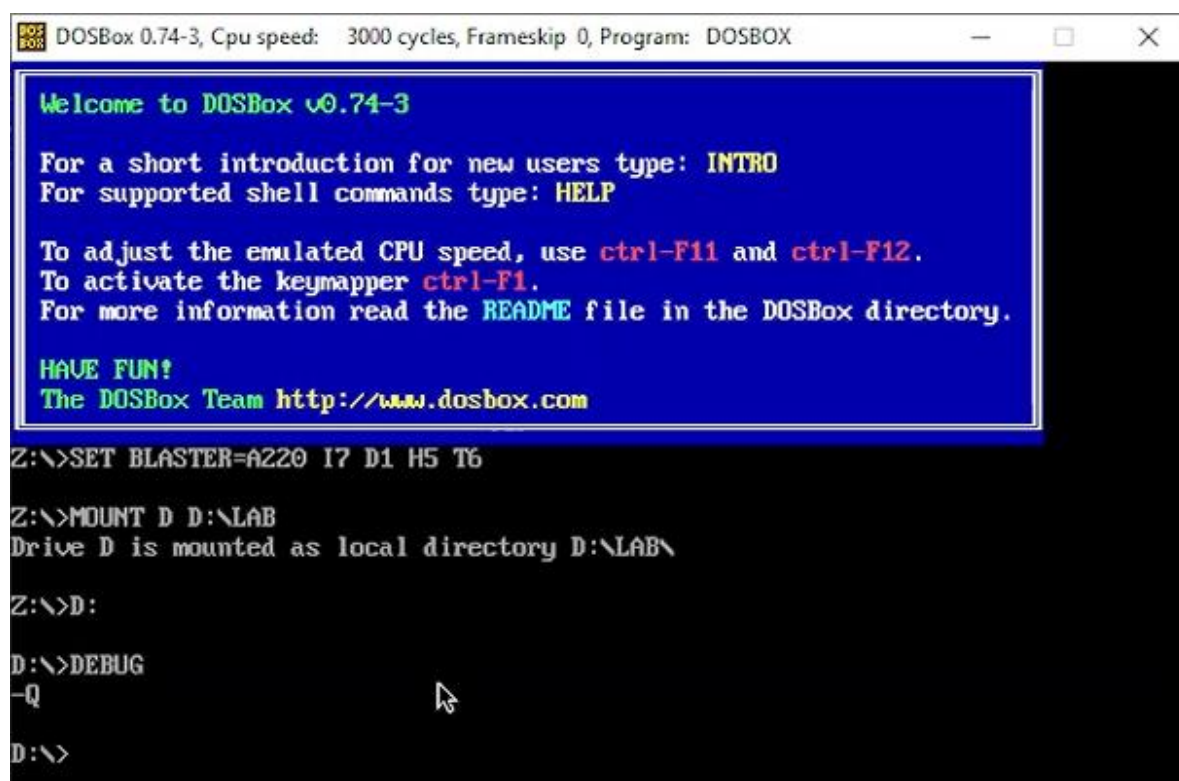
Z:\>SET BLASTER=A220 I7 D1 H5 T6

Z:\>MOUNT D D:\LAB
Drive D is mounted as local directory D:\LAB\

Z:\>D:

D:\>DEBUG
```

Exiting DEBUG



DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DOSBOX

```
Welcome to DOSBox v0.74-3

For a short introduction for new users type: INTRO
For supported shell commands type: HELP

To adjust the emulated CPU speed, use ctrl-F11 and ctrl-F12.
To activate the keymapper ctrl-F1.
For more information read the README file in the DOSBox directory.

HAVE FUN!
The DOSBox Team http://www.dosbox.com

Z:\>SET BLASTER=A220 I7 D1 H5 T6

Z:\>MOUNT D D:\LAB
Drive D is mounted as local directory D:\LAB\

Z:\>D:

D:\>DEBUG
-Q

D:\>
```

Lab Exercise #1

Task 1: Enter D command to display the memory contents of your system and answer the following:

```
DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
To adjust the emulated CPU speed, use ctrl-F11 and ctrl-F12.
To activate the keymapper ctrl-F1.
For more information read the README file in the DOSBox directory.
HAVE FUN!
The DOSBox Team http://www.dosbox.com

Z:\>SET BLASTER=A220 I7 D1 H5 T6

Z:\>MOUNT C C:\LAB
Drive C is mounted as local directory C:\LAB\

Z:\>C:

C:\>DEBUG
-D
073F:0100  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
073F:0110  00 00 00 00 00 00 00 00 00-00 00 00 00 34 00 2E 07  .....4...
073F:0120  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
073F:0130  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
073F:0140  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
073F:0150  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
073F:0160  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
073F:0170  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
```

What is the Code segment address?

073F

The Code Segment (CS) is one of the segment registers in the x86 architecture. It is used to point to the base address of the memory segment containing the currently executing program or code. The CS register, along with the Instruction Pointer (IP), determines the memory address of the instruction currently being executed by the CPU.

What is the content of offset 011Ch?

34

```
-d 011c 011c
073F:0110                                34
```

What is the physical address of the memory location given in above part?

Physical address = (Segment address)(10)H+offset
= 073F(10)+011C
= 73F0+011C
= 750C

Task 2: Do the following and capture each screen shot using print screen option.
Use D command to display the contents of memory from CS:0500 to CS:055F

```
-D CS:0500 055F
073F:0500  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
073F:0510  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
073F:0520  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
073F:0530  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
073F:0540  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
073F:0550  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
```

Use D command to display the contents of memory from 2000:0A20. Also determine what is the final offset address after using D command?

```
-D 2000:0A20
2000:0A20  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
2000:0A30  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
2000:0A40  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
2000:0A50  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
2000:0A60  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
2000:0A70  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
2000:0A80  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
2000:0A90  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
-
```

Final offset address: 0A90

What is the physical address of the memory location 3FCE:0105?

```
3FCE:0105  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
3FCE:0115  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
3FCE:0125  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
3FCE:0135  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
3FCE:0145  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
3FCE:0155  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
3FCE:0165  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
3FCE:0175  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
```

Physical address = (Segment address)(10)H+offset
 = 3FCE(10)+ 0105
 = 3FCE0+0105
 = 3FDE5

Task 3: Use E command to change the contents of memory locations from the code segment, for offsets from 0700 to 0709 to have ASCII contents from 0 to 9.

```
-E 0700 30 31 32 34 36 37 38 39
-D 0700
073F:0700  30 31 32 34 36 37 38 39-00 00 00 00 00 00 00 00  01246789.....
073F:0710  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
073F:0720  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
073F:0730  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
073F:0740  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
073F:0750  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
073F:0760  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
073F:0770  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00  .....
```

In this command:

- `E` is the edit command.
- `0700` is the starting offset within the code segment.
- `30 31 32 33 34 35 36 37 38 39` are the ASCII values for '0' to '9'.

Task 4: Do the following using Memory management commands and capture each screen using print screen option.

Display the memory contents starting from memory location 0100h

```
-D 0100
073F:0100  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
073F:0110  00 00 00 00 00 00 00 00 00-00 00 00 00 34 00 2E 07  .....4...
073F:0120  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
073F:0130  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
073F:0140  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
073F:0150  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
073F:0160  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
073F:0170  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
```

Display the memory contents of first 10 bytes starting from memory location 0100h

```
-d 0100 0109
073F:0100  00 00 00 00 00 00 00 00 00-00 00
```

```
-d 0100 1a
073F:0100  00 00 00 00 00 00 00 00 00-00 00
```

Edit the memory location starting from 0200h with your name and 3-digit Roll #

```
-E 0200 46 61 69 73 61 6C 20 31 35 32
-D 0200
073F:0200  46 61 69 73 61 6C 20 31-35 32 00 00 00 00 00 00 00  Faisal 152.....
073F:0210  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
073F:0220  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
073F:0230  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
073F:0240  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
073F:0250  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
073F:0260  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
073F:0270  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
```

In this command:

- `E` is the edit command.
- `0200` is the starting offset.
- `46` is the ASCII value for 'F'.
- `61` is the ASCII value for 'a'.
- `69` is the ASCII value for 'i'.
- `73` is the ASCII value for 's'.
- `61` is the ASCII value for 'a'.
- `6C` is the ASCII value for 'l'.
- `20` is the ASCII value for a space.
- `31` is the ASCII value for '1'.
- `35` is the ASCII value for '5'.
- `32` is the ASCII value for '2'.

Note the last offset (memory) value of the last digit of your roll #

The last digit of my roll number '2' is located at memory offset 0209h.

Move the block of memory where you filled your name and Roll # to the memory location starting from 0400h

```
-M 0200 0209, 0400
-D 0400
073F:0400  46 61 69 73 61 6C 20 31-35 32 00 00 00 00 00 00 00  Faisal 152.....
073F:0410  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
073F:0420  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
073F:0430  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
073F:0440  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
073F:0450  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
073F:0460  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
073F:0470  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
```

Compare the two memory blocks (one starting from 0200h & the other from 0400h)

```
-C 0200 0209, 0400
-
```

Contents of both memory locations are same

Edit 20XX-CS-to the memory block starting from 0400h before your Roll #

```
-E 0400 '2020-CS-152 Faisal'
-D 0400
073F:0400  32 30 32 30 2D 43 53 2D-31 35 32 20 46 61 69 73  2020-CS-152 Fais
073F:0410  61 6C 00 00 00 00 00 00-00 00 00 00 00 00 00 00  al.....
073F:0420  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
073F:0430  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
073F:0440  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
073F:0450  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
073F:0460  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
073F:0470  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
```

Compare the two memory blocks (one starting from 0200h & the other from 0400h)

```
-C 0200 0209, 0400
073F:0200  46 32 073F:0400
073F:0201  61 30 073F:0401
073F:0202  69 32 073F:0402
073F:0203  73 30 073F:0403
073F:0204  61 2D 073F:0404
073F:0205  6C 43 073F:0405
073F:0206  20 53 073F:0406
073F:0207  31 2D 073F:0407
073F:0208  35 31 073F:0408
073F:0209  32 35 073F:0409
```

Lab Exercise #2

Task 1: Assemble and Unassemble the following code and fill the following table.

```
-a 100
072A:0100 MOV AX, 0200
072A:0103 MOV BX, 0400
072A:0106 ADD AX, BX
072A:0108
```

```
-u 100 106
072A:0100 B80002      MOV     AX,0200
072A:0103 BB0004      MOV     BX,0400
072A:0106 01D8        ADD     AX,BX
```

Logical Address	Opcode	Assembly code	Comments
072A: 0100	B80002	MOV AX,0200H	; AX=0200
072A:0103	BB0004	MOV BX, 0400	; BX=0400
072A:0106	01D8	ADD AX, BX	; AX=AX+BX ; AX=0400+0200

Task 02: Give answers to the following questions after unassemble the code: Why the values 0200H and 0400H are written as 0002 and 0004, in the opcode, respectively?

Due to the little-endian convention lower byte is filled in the lower memory and higher byte is filled in the higher memory.

Write the number of bytes and the value of IP register taken by each instruction.

Instruction	IP Value	Memory
MOV AX,0200	0100	3 Bytes
MOV AX,0400	0103	3 Bytes
ADD AX, BX	0106	2 Bytes

Task 03:

Assemble a program using the DEBUG programming utility to move the decimal values in the registers, as given below:

AX=5431
BX = 9321
CX=45032
DX=23102

(Hint: First convert the above numbers in Hexadecimal system)

DECIMAL TO HEXADECIMAL CONVERSION:

AX= 5431→1537

Result	Remainder
5431/16 = 339.4375	.4376*16 = 7
339/16 = 21.1875	.1875*16 = 3
21/16 = 1.3125	.3125*16 = 5
1	1

BX=9321→2469

Result	Remainder
9321/16 = 582.5625	.5625*16 = 9
582/16 = 36.375	.375*16 = 6
36/16 = 2.25	.25*16 = 4
2	2

CX= 45032→AFE8

Result	Remainder
45032/16 = 2814.5	.5*16 = 8
2814/16 = 175.875	.875*16 = E
175/16 = 10.9375	.9375*16 = F
10 → A	A

DX= 23102→5A3E

Result	Remainder
23102/16 = 1443.875	.875*16 = E
1443/16 = 90.1875	.1875*16 = 3
90/16 = 5.625	.625*16 = A
5	5

```
-a 200
072A:0200 MOV AX,1537
072A:0203 MOV BX,2469
072A:0206 MOV CX,AFE8
072A:0209 MOV DX,5A3E
```

```
-u 200 209
072A:0200 B83715      MOV     AX,1537
072A:0203 BB6924      MOV     BX,2469
072A:0206 B9EBAF      MOV     CX,AFE8
072A:0209 BA3E5A      MOV     DX,5A3E
```

Task 04: Apply the program control command T to execute the code give in Activity -1 & 3. Capture the screenshot. Write down and analyze the values of each registers including IP and Flag registers.

Task#1:

```
-t 3
AX=0200 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=072A ES=072A SS=072A CS=072A IP=0103 NU UP EI NG NZ NA PO NC
072A:0103 BB0004          MOV     BX,0400
AX=0200 BX=0400 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=072A ES=072A SS=072A CS=072A IP=0106 NU UP EI NG NZ NA PO NC
072A:0106 01D8          ADD     AX,BX
AX=0600 BX=0400 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=072A ES=072A SS=072A CS=072A IP=0108 NU UP EI PL NZ NA PE NC
072A:0108 0000          ADD     [BX+SI],AL          DS:0400=00
```

REGISTER VALUES:

Step#01:
AX=0200, BX=0000, CX=0000, DX=0000
Step#02:
AX=0200, BX=0400, CX=0000, DX=0000
Step#03:
AX=0600, BX=0400, CX=0000, DX=0000

IP VALUES:

Step#01:
103
Step# 02:
106
Step#03:
108

FLAG REGISTER VALUES:

Flags	Output Step 1	Output Step 2	Output Step 3
Overflow Flag	0	0	0
Direction Flag	0	0	0
Interrupt Flag	1	1	1
Sign Flag	1	1	0
Zero Flag	0	0	0
Auxiliary Carry Flag	0	0	0
Parity Flag	0	0	1
Carry Flag	0	0	0

TASK#03:

To Check 200 Offset Value We First Have to Change the Value Of Instruction Pointer To 200.

```
-r ip
IP 0100 :200
-t4
AX=1537 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=072A ES=072A SS=072A CS=072A IP=0203 NU UP EI NG NZ NA PO NC
072A:0203 BB6924      MOV     BX,2469
AX=1537 BX=2469 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=072A ES=072A SS=072A CS=072A IP=0206 NU UP EI NG NZ NA PO NC
072A:0206 B9E8AF      MOV     CX,AFE8
AX=1537 BX=2469 CX=AFE8 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=072A ES=072A SS=072A CS=072A IP=0209 NU UP EI NG NZ NA PO NC
072A:0209 BA3E5A      MOV     DX,5A3E
AX=1537 BX=2469 CX=AFE8 DX=5A3E SP=FFFE BP=0000 SI=0000 DI=0000
DS=072A ES=072A SS=072A CS=072A IP=020C NU UP EI NG NZ NA PO NC
072A:020C 0000      ADD     [BX+SI],AL      DS:2469=00
```

REGISTER VALUES:

Step#01:

AX=1537, BX=0000, CX=0000,DX=0000

Step#02:

AX=1537,BX-2469, CX=0000, DX=0000

Step#03:

AX=1537,BX-2469, CX=AFE8,DX=0000

Step#04:

AX=1537,BX-2469, CX=AFE8,DX=5A3E

IP VALUES:

Step#01:

0203

Step#02:

0206

Step#03:

0209

Step# 04:

020C

FLAG REGISTER VALUES:

Flags	Output Step 1	Output Step 2	Output Step 3
Overflow Flag	0	0	0
Direction Flag	0	0	0
Interrupt Flag	1	1	1
Sign Flag	1	1	1
Zero Flag	0	0	0
Auxiliary Carry Flag	0	0	0
Parity Flag	0	0	0
Carry Flag	0	0	0

Task 5: Apply the program control command G to execute the code give in Activity -1 & 3. Capture the screenshot. Write down and analyze the values of each registers including IP and Flag registers.

Task#1:

```
AX=0600 BX=0400 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=072A ES=072A SS=072A CS=072A IP=0108 NV UP EI PL NZ NA PE NC
072A:010B 0000      ADD     [BX+SI],AL      DS:0400=00
```

Register Values:

AX=600, BX=-0400

IP VALUE:

0108

FLAG REGISTER VALUES:

Flags	Output
Overflow Flag	0
Direction Flag	0
Interrupt Flag	1
Sign Flag	0
Zero Flag	0
Auxiliary Carry Flag	0
Parity Flag	1
Carry Flag	0

Task#3:

```
AX=1537 BX=2469 CX=AFE8 DX=5A3E SP=FFFE BP=0000 SI=0000 DI=0000
DS=072A ES=072A SS=072A CS=072A IP=020C NV UP EI NG NZ NA PD NC
072A:020C 0000      ADD     [BX+SI],AL      DS:2469=00
```

Register Values:

AX=1537, BX=2469, CX=AFE8, DX=5A3E

IP VALUE:

020C

FLAG REGISTER VALUES:

Flags	Output
Overflow Flag	0
Direction Flag	0
Interrupt Flag	1
Sign Flag	1
Zero Flag	0
Auxiliary Carry Flag	0
Parity Flag	0
Carry Flag	0

Lab Exercise #3

Task 1: Consider the following 8 bytes of data:

LFh, 45h, 89h, 4Ch, 2Ah, 58h, 64h, 37h

Write a program to store the given block of data bytes in memory locations starting from 0200h.

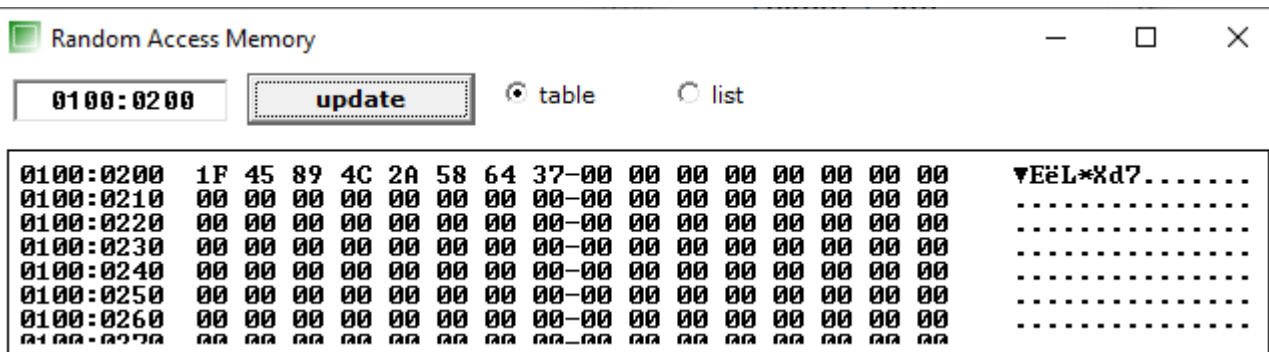
List the addressing modes used in the process. Use the debug option and observe the data. Attach the image with your solution.

Code:

```
mov [0200h],1fh
mov [0201h],45h
mov [0202h],89h
mov [0203h],4ch
mov [0204h],2ah
mov [0205h],58h
mov [0206h],64h
mov [0207h],37h
```

Addressing mode: Direct addressing mode

Output:



Task 2: Write a program to move the block of data, given in Activity-1, from locations 0200-0207 to memory locations 0400h -0407h using direct addressing mode. Run the program step- by-step and analyze each step. Attach the image with your solution.

Code:

```
mov al,[0200h]
mov [0400h],al
```

```
mov al,[0201h]
mov [0401h],al
```

```
mov al,[0202h]
mov [0402h],al
```

```
mov al,[0203h]
mov [0403h],al
```

```
mov al,[0204h]
mov [0404h],al
```

```
mov bx,0205h
mov al,[bx]
mov [0405h],al
```

```

mov bx,0206h
mov al,[bx]
mov [0406h],al

```

```

mov bx,0207h
mov al,[bx]
mov [0407h],al

```

Addressing mode: Register indirect addressing mode

Output:

Random Access Memory

0100:0400

update

☒ table
☐ list

0100:0400	1F 45 89 4C 2A 58 64 37-00 00 00 00 00 00 00 00	▼EëL*Xd7.....
0100:0410	00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00	
0100:0420	00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00	
0100:0430	00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00	
0100:0440	00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00	
0100:0450	00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00	
0100:0460	00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00	
0100:0470	00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00	

debug log - debug.exe emulation

clear

save to file...

?

AX=0037 BX=0207 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000

DS=0100 ES=0100 SS=0100 CS=0100 IP=0068 NV UP EI PL NZ NA PO NC

0100:0068 F4 HLT

Task 4: Write a program to swap the bytes from locations 0501h, 0503h, 0505h, and 0507h with the locations 0701h, 0703h, 0705h, 0707h respectively using

- Based-relative addressing mode
- Indexed-relative addressing mode

Based-relative addressing mode

Code:

```

mov bx,0501h
mov al,[bx]

```

```

mov bp,0701h
mov dl,[bp]

```

```

xchg al,dl

```

```

mov [0501h],al
mov [0701h],al

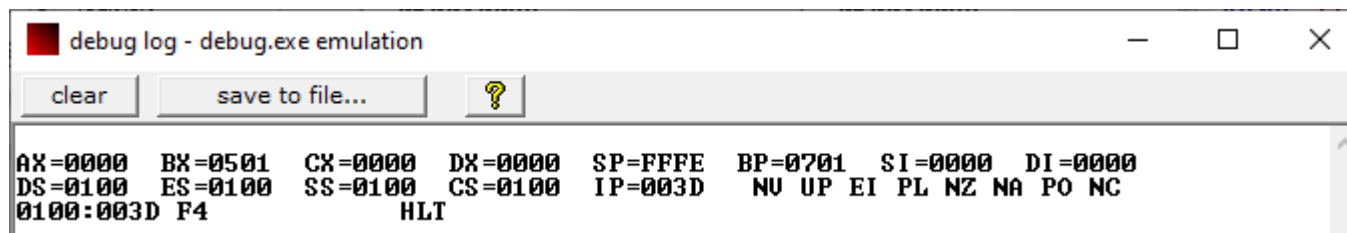
```



```
mov al,[bx]+2
mov dl,[bp]+2
xchg al,dl
mov [0503h],al
mov [0703h],al
```

```
mov al,[bx]+4
mov dl,[bp]+4
xchg al,dl
mov [0505h],al
mov [0705h],al
```

```
mov al,[bx]+6
mov dl,[bp]+6
xchg al,dl
mov [0507h],al
mov [0707h],al
```



Index-relative addressing mode

Code:

```
mov si,0501h
mov al,[si]

mov di,0701h
mov dl,[di]

xchg al,dl

mov [0501h],al
mov [0701h],al

mov al,[si]+2
mov dl,[di]+2
xchg al,dl
mov [0503h],al
mov [0703h],al

mov al,[si]+4
mov dl,[di]+4
xchg al,dl
```

```
mov [0505h],al
mov [0705h],al
```

```
mov al,[si]+6
mov dl,[di]+6
xchg al,dl
mov [0507h],al
mov [0707h],al
```

debug log - debug.exe emulation

clear

save to file...

?

AX=0000 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0501 DI=0701
DS=0100 ES=0100 SS=0100 CS=0100 IP=003C NU UP EI PL NZ NA PO NC
0100:003C F4 HLT

Lab Exercise #4

Task 1: Write a program to clear the screen, set the cursor at the beginning of the fifth line from the top of the screen, and display "We are the students of Computer Science & Information Technology Department!"

Code:

```
TITLE LAB 4 ACTIVITY 1
```

```
.MODEL SMALL
```

```
.STACK 100H
```

```
.DATA
```

```
MSG DB "FAISAL IS A STUDENT OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY DEPARTMENT!$"
```

```
.CODE
```

```
MAIN PROC
```

```
    ;INITIATE DATA SEGMENT
```

```
    MOV AX,@DATA
```

```
    MOV DS,AX
```

```
    ;CODE TO CLEAR SCREEN
```

```
    MOV AX,0600H
```

```
    MOV BH,07
```

```
    MOV CX,0000
```

```
    MOV DX,184FH
```

```
    INT 10H
```

```
    ;CODE TO SET THE CURSOR POSITION TO ROW=05H AND COLUMN=00H
```

```
    MOV AH,2
```

```
    MOV BH,00
```

```
    MOV DX,0500H
```

```
    INT 10H
```

```
    ;CODE TO DISPLAY STRING
```

```
    MOV AH,9
```

```
    MOV DX,OFFSET MSG
```

```
    INT 21H
```

```
    ;EXIT TO DOS
```

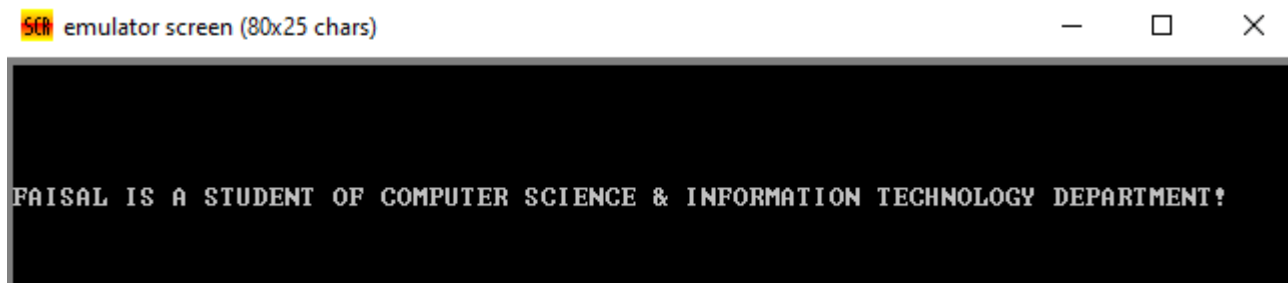
```
    MOV AH,4CH
```

```
    INT 21H
```

```
MAIN ENDP
```

```
    END MAIN
```

Output:



Task 2: Write a program to get the first name of the user by displaying the message "Enter your first name:" on screen. Store it in a variable named "FName", and display the entered name, character by character, in the next line with the message "Your first name is ...".

Code:

TITLE LAB 4 ACTIVITY 2

.MODEL SMALL

.STACK 100H

.DATA

MSG1 DB "ENTER YOUR FIRST NAME: \$"

FNAME DB 12 DUP('\$')

MSG2 DB "YOUR FIRST NAME IS: \$"

.CODE

MAIN PROC

;INITIATE DATA SEGMENT

MOV AX,@DATA

MOV DS,AX

;DISPLAY MSG1

MOV AH,9

MOV DX,OFFSET MSG1

INT 21H

;COPY CONTENT OF FNAME

MOV SI,OFFSET FNAME

;TAKE INPUT

INPUT:

MOV AH,1

INT 21H

CMP AL,0DH

JE PRINT

MOV [SI],AL

INC SI

JMP INPUT

PRINT:

MOV DL,0DH

MOV AH,2

INT 21H

MOV DL,0AH

INT 21H

;DISPLAY MSG2

MOV AH,9

MOV DX,OFFSET MSG2

INT 21H

```

;COPY CONTENT OF FNAME
MOV SI,OFFSET FNAME

;DISPLAY THE ENTERED NAME CHARACTER BY CHARACTER
DISPLAY:
MOV DL,[SI]
CMP DL,'$'

JNZ OK
JMP EXIT

OK:
MOV AH,2
INT 21H
INC SI
JMP DISPLAY

;EXIT TO DOS
EXIT:
MOV AH,4CH
INT 21H

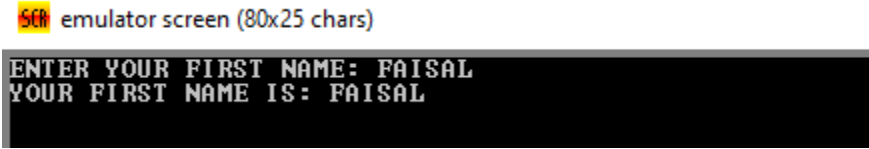
```

```

MAIN ENDP
END MAIN

```

Output:

 emulator screen (80x25 chars)

```

ENTER YOUR FIRST NAME: FAISAL
YOUR FIRST NAME IS: FAISAL

```

Task 3: Write a program to modify the activity-2 to display the entered name using service 9 of INT 21h.

(Hint: a 'S' character must be appended at the end of first name entered by the user)

Code:

```

TITLE LAB 4 ACTIVITY 3
.MODEL SMALL
.STACK 100H
.DATA

MSG1 DB "ENTER YOUR FIRST NAME: $"
FNAME DB 12 DUP('$')
MSG2 DB "YOUR FIRST NAME IS: $"
.CODE

MAIN PROC
;INITIATE DATA SEGMENT
MOV AX,@DATA
MOV DS,AX

```

```

;DISPLAY MSG1
MOV AH,9
MOV DX,OFFSET MSG1
INT 21H

;COPY CONTENT OF FNAME
MOV SI,OFFSET FNAME

;TAKE INPUT
INPUT:
MOV AH,1
INT 21H
CMP AL,0DH
JE PRINT

MOV [SI],AL
INC SI

JMP INPUT

PRINT:
MOV DL,0DH
MOV AH,2
INT 21H

MOV DL,0AH
INT 21H

;DISPLAY MSG2
MOV AH,9
MOV DX,OFFSET MSG2
INT 21H

;DISPLAY THE ENTERED NAME USING SERVICE 9 OF INT 21H
MOV DX,OFFSET FNAME
INT 21H

;EXIT TO DOS
MOV AH,4CH
INT 21H

MAIN ENDP
END MAIN

```

Output:

 emulator screen (80x25 chars)

```

ENTER YOUR FIRST NAME: FAISAL
YOUR FIRST NAME IS: FAISAL

```

Task 4: Write a program to display the following pattern at the center on the cleared screen.

12345
234
3
432
54321

Code:

```
TITLE LAB 4 ACTIVITY 4
.MODEL SMALL
.STACK 100H
.DATA
```

```
LINE1 DB "12345 $"
LINE2 DB "234 $"
LINE4 DB "432 $"
LINE5 DB "54321 $"
```

```
.CODE
```

```
MAIN PROC
```

```
;INITIATE DATA SEGMENT
MOV AX,@DATA
MOV DS,AX
```

```
;CLEAR SCREEN
MOV AX,0600H
MOV BH,07
MOV CX,0000
MOV DX,184FH
INT 10H
```

```
;SET THE CURSOR POSITION TO ROW=0AH AND COLUMN=26H
MOV AH,2
MOV BH,00
MOV DX,0A26H
INT 10H
```

```
;DISPLAY LINE1
MOV AH,9
MOV DX,OFFSET LINE1
INT 21H
```

```
;SET THE CURSOR POSITION TO ROW=0BH AND COLUMN=27H
MOV AH,2
MOV BH,00
MOV DX,0B27H
INT 10H
```

```
;DISPLAY LINE2
MOV AH,9
MOV DX,OFFSET LINE2
```

INT 21H

;SET THE CURSOR POSITION TO ROW=0CH AND COLUMN=28H

MOV AH,2

MOV BH,00

MOV DX,0C28H

INT 10H

;DISPLAY CHARACTER '3'

MOV AH,2

MOV DL,'3'

INT 21H

;SET THE CURSOR POSITION TO ROW=0DH AND COLUMN=27H

MOV AH,2

MOV BH,00

MOV DX,0D27H

INT 10H

;DISPLAY LINE4

MOV AH,9

MOV DX,OFFSET LINE4

INT 21H

;SET THE CURSOR POSITION TO ROW=0EH AND COLUMN=26H

MOV AH,2

MOV BH,00

MOV DX,0E26H

INT 10H

;DISPLAY LINE5

MOV AH,9

MOV DX,OFFSET LINE5

INT 21H

;EXIT TO DOS

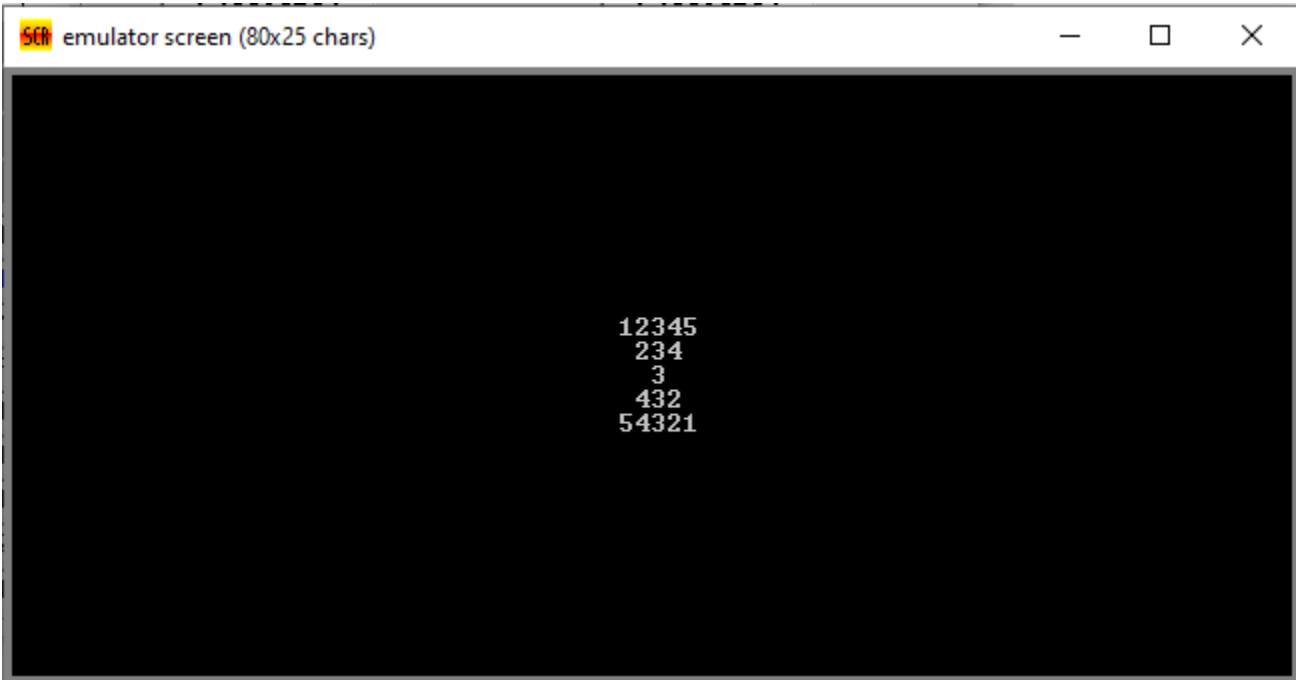
MOV AH,4CH

INT 21H

MAIN ENDP

END MAIN

Output:



Lab Exercise #5

Task 1: Write a program to display the first letter of your name 10 times using jump instruction. For example, your name starts with "A" then the output will be:

AAAAAA

Code:

```
TITLE LAB 5 ACTIVITY 1
```

```
.MODEL SMALL
```

```
.STACK 100H
```

```
.CODE
```

```
MAIN PROC
```

```
;Initialize CL with the count (10)
```

```
MOV CL, 10
```

```
;Initialize DL with the character 'F'
```

```
MOV DL, 'F'
```

```
;Function code for displaying a character
```

```
MOV AH, 2
```

```
;Loop to display 'F' 10 times
```

```
LOOP1:
```

```
;Display the character in DL
```

```
INT 21H
```

```
;Decrement the count in CL
```

```
DEC CL
```

```
;Check if the count is not zero, and if not, repeat the loop
```

```
JNZ LOOP1
```

```
;Exit to DOS
```

```
MOV AH, 4CH
```

```
INT 21H
```

```
MAIN ENDP
```

```
END MAIN
```

Output:

 emulator screen (80x25 chars)

```
FFFFFFFFFF
```

Task 2: Write a program to display all the ASCII characters on screen line by line. Make sure to display character screen-by-screen.

(Hint: After displaying first 23 characters, ask user to continue by providing the statement "press any key to continue..." to display next 23 characters and so on.)

Code:

TITLE LAB 5 ACTIVITY 2

.MODEL SMALL

.STACK 100H

.DATA

PROMPT DB 'THE 256 ASCII CHARACTER ARE: \$'

CONTINUE DB 'PRESS ANY KEY TO CONTINUE... \$'

.CODE

MAIN PROC

; INITIATE DATA SEGMENT

MOV AX, @DATA

MOV DS, AX

; DISPLAY PROMPT

MOV AH, 9

MOV DX, OFFSET PROMPT

INT 21H

; NEXT LINE

MOV AH, 2

MOV DL, 0AH

INT 21H

; MOVE THE CURSOR TO THE BEGINNING OF THE CURRENT LINE

MOV DL, 0DH

INT 21H

MOV CX, 23

MOV DL, 0

LOOP1:

MOV AH, 2

INT 21H

INC DL

DEC CX

JNZ LOOP1

MOV AH, 2

MOV DL, 0AH

INT 21H

MOV DL, 0DH

INT 21H

; DISPLAY CONTINUE MESSAGE

```
MOV AH, 9
MOV DX, OFFSET CONTINUE
INT 21H
```

```
; WAIT FOR USER INPUT
MOV AH, 1
INT 21H
```

```
; CLEAR SCREEN
MOV AH, 6
MOV AL, 0
MOV BH, 7
MOV CX, 0
MOV DX, 184FH
INT 10H
```

```
MOV CX, 23
MOV DL, 18H
```

```
LOOP2:
MOV AH, 2
INT 21H
INC DL
DEC CX
JNZ LOOP2
```

```
MOV AH, 2
MOV DL, 0AH
INT 21H
MOV DL, 0DH
INT 21H
```

```
; DISPLAY CONTINUE MESSAGE
MOV AH, 9
MOV DX, OFFSET CONTINUE
INT 21H
```

```
; WAIT FOR USER INPUT
MOV AH, 1
INT 21H
```

```
; CLEAR SCREEN
MOV AH, 6
MOV AL, 0
MOV BH, 7
MOV CX, 0
MOV DX, 184FH
INT 10H
```

```
MOV CX, 23
MOV DL, 2FH
```

LOOP3:

```
MOV AH, 2
INT 21H
INC DL
DEC CX
JNZ LOOP3
```

```
MOV AH, 2
MOV DL, 0AH
INT 21H
MOV DL, 0DH
INT 21H
```

```
; DISPLAY CONTINUE MESSAGE
MOV AH, 9
MOV DX, OFFSET CONTINUE
INT 21H
```

```
; WAIT FOR USER INPUT
MOV AH, 1
INT 21H
```

```
; CLEAR SCREEN
MOV AH, 6
MOV AL, 0
MOV BH, 7
MOV CX, 0
MOV DX, 184FH
INT 10H
```

```
MOV CX, 23
MOV DL, 46H
```

LOOP4:

```
MOV AH, 2
INT 21H
INC DL
DEC CX
JNZ LOOP4
```

```
MOV AH, 2
MOV DL, 0AH
INT 21H
MOV DL, 0DH
INT 21H
```

```
; DISPLAY CONTINUE MESSAGE
MOV AH, 9
MOV DX, OFFSET CONTINUE
INT 21H
```

```
; WAIT FOR USER INPUT
```

```
MOV AH, 1  
INT 21H
```

```
; CLEAR SCREEN  
MOV AH, 6  
MOV AL, 0  
MOV BH, 7  
MOV CX, 0  
MOV DX, 184FH  
INT 10H
```

```
MOV CX, 23  
MOV DL, 5DH
```

```
LOOP5:  
MOV AH, 2  
INT 21H  
INC DL  
DEC CX  
JNZ LOOP5
```

```
MOV AH, 2  
MOV DL, 0AH  
INT 21H  
MOV DL, 0DH  
INT 21H
```

```
; DISPLAY CONTINUE MESSAGE  
MOV AH, 9  
MOV DX, OFFSET CONTINUE  
INT 21H
```

```
; WAIT FOR USER INPUT  
MOV AH, 1  
INT 21H
```

```
; CLEAR SCREEN  
MOV AH, 6  
MOV AL, 0  
MOV BH, 7  
MOV CX, 0  
MOV DX, 184FH  
INT 10H
```

```
MOV CX, 23  
MOV DL, 74H
```

```
LOOP6:  
MOV AH, 2  
INT 21H  
INC DL  
DEC CX
```

JNZ LOOP6

MOV AH, 2
MOV DL, 0AH
INT 21H
MOV DL, 0DH
INT 21H

; DISPLAY CONTINUE MESSAGE
MOV AH, 9
MOV DX, OFFSET CONTINUE
INT 21H

; WAIT FOR USER INPUT
MOV AH, 1
INT 21H

; CLEAR SCREEN
MOV AH, 6
MOV AL, 0
MOV BH, 7
MOV CX, 0
MOV DX, 184FH
INT 10H

MOV CX, 23
MOV DL, 8BH

LOOP7:

MOV AH, 2
INT 21H
INC DL
DEC CX
JNZ LOOP7

MOV AH, 2
MOV DL, 0AH
INT 21H
MOV DL, 0DH
INT 21H

; DISPLAY CONTINUE MESSAGE
MOV AH, 9
MOV DX, OFFSET CONTINUE
INT 21H

; WAIT FOR USER INPUT
MOV AH, 1
INT 21H

; CLEAR SCREEN
MOV AH, 6

```
MOV AL, 0
MOV BH, 7
MOV CX, 0
MOV DX, 184FH
INT 10H
```

```
MOV CX, 23
MOV DL, 0A2H
```

LOOP8:

```
MOV AH, 2
INT 21H
INC DL
DEC CX
JNZ LOOP8
```

```
MOV AH, 2
MOV DL, 0AH
INT 21H
MOV DL, 0DH
INT 21H
```

; DISPLAY CONTINUE MESSAGE

```
MOV AH, 9
MOV DX, OFFSET CONTINUE
INT 21H
```

; WAIT FOR USER INPUT

```
MOV AH, 1
INT 21H
```

; CLEAR SCREEN

```
MOV AH, 6
MOV AL, 0
MOV BH, 7
MOV CX, 0
MOV DX, 184FH
INT 10H
```

```
MOV CX, 23
MOV DL, 0B8H
```

LOOP9:

```
MOV AH, 2
INT 21H
INC DL
DEC CX
JNZ LOOP9
```

```
MOV AH, 2
MOV DL, 0AH
INT 21H
```



```
MOV DL, 0DH  
INT 21H
```

```
; DISPLAY CONTINUE MESSAGE  
MOV AH, 9  
MOV DX, OFFSET CONTINUE  
INT 21H
```

```
; WAIT FOR USER INPUT  
MOV AH, 1  
INT 21H
```

```
; CLEAR SCREEN  
MOV AH, 6  
MOV AL, 0  
MOV BH, 7  
MOV CX, 0  
MOV DX, 184FH  
INT 10H
```

```
MOV CX, 23  
MOV DL, 0B9H
```

```
LOOP10:  
MOV AH, 2  
INT 21H  
INC DL  
DEC CX  
JNZ LOOP10
```

```
MOV AH, 2  
MOV DL, 0AH  
INT 21H  
MOV DL, 0DH  
INT 21H
```

```
; DISPLAY CONTINUE MESSAGE  
MOV AH, 9  
MOV DX, OFFSET CONTINUE  
INT 21H
```

```
; WAIT FOR USER INPUT  
MOV AH, 1  
INT 21H
```

```
; CLEAR SCREEN  
MOV AH, 6  
MOV AL, 0  
MOV BH, 7  
MOV CX, 0  
MOV DX, 184FH  
INT 10H
```

```
MOV CX, 23
MOV DL, 0D0H
```

LOOP11:

```
MOV AH, 2
INT 21H
INC DL
DEC CX
JNZ LOOP11
```

```
MOV AH, 2
MOV DL, 0AH
INT 21H
MOV DL, 0DH
INT 21H
```

```
; DISPLAY CONTINUE MESSAGE
MOV AH, 9
MOV DX, OFFSET CONTINUE
INT 21H
```

```
; WAIT FOR USER INPUT
MOV AH, 1
INT 21H
```

```
; CLEAR SCREEN
MOV AH, 6
MOV AL, 0
MOV BH, 7
MOV CX, 0
MOV DX, 184FH
INT 10H
```

```
MOV CX, 23
MOV DL, 0E7H
```

LOOP12:

```
MOV AH, 2
INT 21H
INC DL
DEC CX
JNZ LOOP12
```

```
MOV AH, 2
MOV DL, 0AH
INT 21H
MOV DL, 0DH
INT 21H
```

```
; DISPLAY CONTINUE MESSAGE
MOV AH, 9
```

```
MOV DX, OFFSET CONTINUE
INT 21H
```

```
; WAIT FOR USER INPUT
MOV AH, 1
INT 21H
```

```
; CLEAR SCREEN
MOV AH, 6
MOV AL, 0
MOV BH, 7
MOV CX, 0
MOV DX, 184FH
INT 10H
```

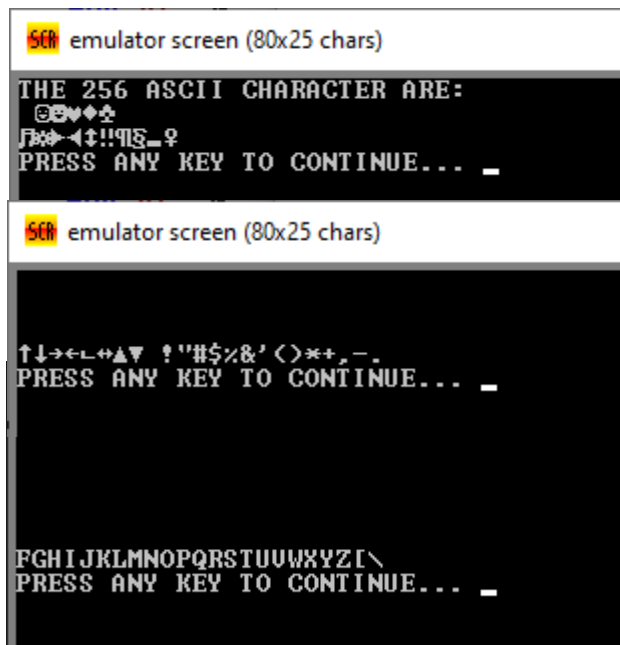
```
MOV CX, 3
MOV DL, 0FEH
```

```
LOOP13:
MOV AH, 2
INT 21H
INC DL
DEC CX
JNZ LOOP13
```

```
;EXIT TO DOS
MOV AH,4CH
INT 21H
```

```
MAIN ENDP
END MAIN
```

Output:



Scr emulator screen (80x25 chars)

```
/0123456789:;<=>?@ABCDE  
PRESS ANY KEY TO CONTINUE... _
```

Scr emulator screen (80x25 chars)

```
l^_`abcdefghijklmnopqrs  
PRESS ANY KEY TO CONTINUE...
```

Scr emulator screen (80x25 chars)

```
tuvwxyz<|>^~`üéâäåçêëè  
PRESS ANY KEY TO CONTINUE... _
```

Scr emulator screen (80x25 chars)

```
iiia8ÉæfôûðûüÿÜÜçFVRfáí  
PRESS ANY KEY TO CONTINUE...
```

Scr emulator screen (80x25 chars)

```
óúññÑº¿-~¼½¿<0>»||+||ñ  
PRESS ANY KEY TO CONTINUE... _
```


Task 3: Write a program to display the first letter of your name, using the character "*", at the center of screen. For example, if the first letter of your name is S, then following pattern should be displayed:

```
*****
*
*
*****
*
*
*
```

Code:

```
TITLE LAB 5 ACTIVITY 3
.MODEL SMALL
.STACK 100H
.CODE

MAIN PROC
; SET VIDEO MODE 2 (MONOCHROME)
MOV AX, 2
INT 10H

; SET DISPLAY PAGE AND CURSOR POSITION
MOV DX, 0727H
MOV AH, 2
INT 10H

; INITIALIZE LOOP COUNTER
MOV CX, 9

LOOP1:
; DISPLAY '*'
MOV AH, 2
MOV DL, '*'
INT 21H

; MOVE TO THE NEXT LINE
MOV AH, 2
MOV DL, 0AH
INT 21H

; MOVE CURSOR TO THE BEGINNING OF THE LINE
MOV AH, 2
MOV DL, 08H
INT 21H

; DECREMENT LOOP COUNTER
DEC CX
```

```
; REPEAT LOOP UNTIL CX BECOMES ZERO  
JNZ LOOP1
```

```
; SET DISPLAY PAGE AND CURSOR POSITION AGAIN  
MOV DX, 0727H  
MOV AH, 2  
INT 10H
```

```
; SET A NEW LOOP COUNTER  
MOV CX, 7
```

LOOP2:

```
; DISPLAY '*'  
MOV AH, 2  
MOV DL, '*'  
INT 21H
```

```
; DECREMENT LOOP COUNTER  
DEC CX
```

```
; REPEAT LOOP UNTIL CX BECOMES ZERO  
JNZ LOOP2
```

```
; SET DISPLAY PAGE AND CURSOR POSITION AGAIN  
MOV DX, 0B27H  
MOV AH, 2  
INT 10H
```

```
; SET A NEW LOOP COUNTER  
MOV CX, 6
```

LOOP3:

```
; DISPLAY '*'  
MOV AH, 2  
MOV DL, '*'  
INT 21H
```

```
; DECREMENT LOOP COUNTER  
DEC CX
```

```
; REPEAT LOOP UNTIL CX BECOMES ZERO  
JNZ LOOP3
```

```
; EXIT TO DOS  
MOV AH, 4CH  
INT 21H
```

```
MAIN ENDP  
END MAIN
```

Output:



Lab Exercise #5

Task 4: Fill out the missing values, using target address calculation, in the following table.

IP	Opcode	Code	
E06C	733F		JNC CODE1
???			MOV AX, 3456H
.....			
E072	7139		JNO CODE1
???			MOV CX, 100H
.....			
???	8ED8	C8:	MOV DS, AX
.....			
E0A7	EBE3		JMP C8
???			CMP AX
.....			
???	F4	CODE1:	HLT

Answer:

IP	Opcode	Code	
E06C	733F		JNC CODE1
E06E			MOV AX, 3456H
.....			
E072	7139		JNO CODE1
E074			MOV CX, 100H
.....			
E08C	8ED8	C8:	MOV DS, AX
.....			
E0A7	EBE3		JMP C8
E0A9			CMP AX
.....			
E0AD	F4	CODE1:	HLT

Target address = IP value of next instruction + E3h
 = A9h+E3h
 = 8Ch
 So, it is E08C

Target address = IP value of next instruction + 3Fh
 = 6Eh+3Fh
 = ADh
 So, it is E0AD

Task 2: Write a program to display the numbers in the specific pattern as shown below.

12345
234
3
432
54321

Code:

TITLE LAB 6 ACTIVITY 2

.MODEL SMALL

; Define the memory model as small

.STACK 100H

; Set the stack size to 100H

.CODE

MAIN PROC

; Initialize loop counter and starting character

MOV CX, 5

MOV DL, '1'

LOOP1:

; Display the current character

MOV AH, 2

; DOS function to display character

INT 21H

; Call DOS interrupt

INC DL

; Move to the next character

LOOP LOOP1

; Jump back to LOOP1 if CX is not zero

; Move to the next line

MOV AH, 2

; DOS function to move cursor

MOV DL, 0DH

; Carriage return

INT 21H

; Call DOS interrupt

MOV DL, 0AH

; Line feed

INT 21H

; Call DOS interrupt

MOV DL, 20H

; Space

INT 21H

; Call DOS interrupt

; Initialize loop counter and starting character for the second part

MOV CX, 3

MOV DL, '2'

LOOP2:

; Display the current character

MOV AH, 2

; DOS function to display character

INT 21H

; Call DOS interrupt

INC DL

; Move to the next character

LOOP LOOP2

; Jump back to LOOP2 if CX is not zero

; Move to the next line

MOV AH, 2

; DOS function to move cursor

MOV DL, 0DH

; Carriage return

INT 21H

; Call DOS interrupt

MOV DL, 0AH

; Line feed

```

INT 21H                ; Call DOS interrupt
MOV DL, 20H            ; Space
INT 21H                ; Call DOS interrupt
MOV DL, 20H            ; Space
INT 21H                ; Call DOS interrupt

MOV DL, '3'            ; Display the third part
INT 21H                ; Call DOS interrupt

```

```

; Move to the next line
MOV AH, 2              ; DOS function to move cursor
MOV DL, 0DH            ; Carriage return
INT 21H                ; Call DOS interrupt
MOV DL, 0AH            ; Line feed
INT 21H                ; Call DOS interrupt
MOV DL, 20H            ; Space
INT 21H                ; Call DOS interrupt

```

```

; Initialize loop counter and starting character for the fourth part
MOV CX, 3
MOV DL, '4'

```

LOOP3:

```

; Display the current character
MOV AH, 2              ; DOS function to display character
INT 21H                ; Call DOS interrupt
DEC DL                 ; Move to the previous character

LOOP LOOP3             ; Jump back to LOOP3 if CX is not zero

```

```

; Move to the next line
MOV AH, 2              ; DOS function to move cursor
MOV DL, 0DH            ; Carriage return
INT 21H                ; Call DOS interrupt
MOV DL, 0AH            ; Line feed
INT 21H                ; Call DOS interrupt

```

```

; Initialize loop counter and starting character for the fifth part
MOV CX, 5
MOV DL, '5'

```

LOOP4:

```

; Display the current character
MOV AH, 2              ; DOS function to display character
INT 21H                ; Call DOS interrupt
DEC DL                 ; Move to the previous character

LOOP LOOP4             ; Jump back to LOOP4 if CX is not zero

```

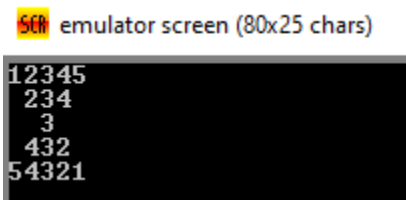
```

; Exit the program
MOV AH, 4CH            ; DOS function to exit program
INT 21H                ; Call DOS interrupt

```

MAIN ENDP	; End of the main procedure
END MAIN	; End of the program with the label MAIN

Output:



Task 3: Write a program to display the numbers in the specific pattern as shown below.

```
1234567
123456
12345
1234
123
12
1
```

Code:

TITLE LAB 6 ACTIVITY 3	
.MODEL SMALL	; Define the memory model as small
.STACK 100H	; Set the stack size to 100H
.CODE	
MAIN PROC	
MOV CX, 7	; Set the outer loop counter
OUTERLOOP:	
PUSH CX	; Save CX on the stack
MOV DL, 31H	; Set the initial character for the inner loop
INNERLOOP:	
MOV AH, 2	; DOS function to display character
INT 21H	; Call DOS interrupt
INC DL	; Move to the next character
LOOP INNERLOOP	; Jump back to INNERLOOP if CX is not zero
MOV AH, 2	; DOS function to move cursor
MOV DL, 0AH	; Line feed
INT 21H	; Call DOS interrupt
MOV DL, 0DH	; Carriage return
INT 21H	; Call DOS interrupt
POP CX	; Restore CX from the stack
LOOP OUTERLOOP	; Jump back to OUTERLOOP if CX is not zero
MOV AH, 4CH	; DOS function to exit program
INT 21H	; Call DOS interrupt
MAIN ENDP	; End of the main procedure

END MAIN

; End of the program with the label MAIN

Output:

Scr emulator screen (80x25 c

1

Scr emulator screen (80x25 c

2

Scr emulator screen (80x25 c

3

Scr emulator screen (80x25 c

4

Scr emulator screen (80x25 c

5

Scr emulator screen (80x25 c

6

Scr emulator screen (80x25 c

7

Scr emulator screen (80x25 c

8

Scr emulator screen (80x25 c

9

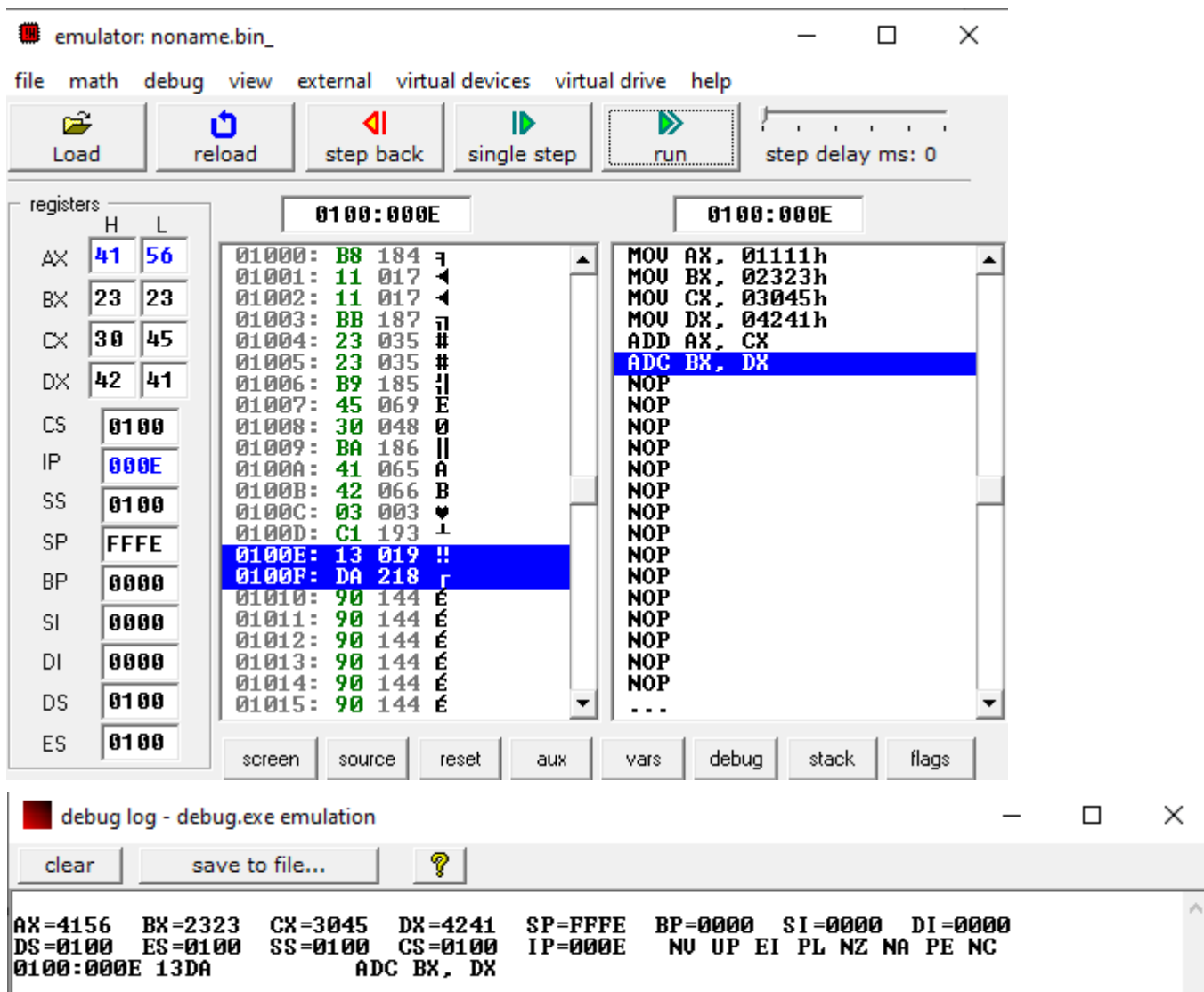
Lab Exercise #7

Task 1: Program to add two 32-bit numbers 23231111h and 42413045h, stored in BX:AX and DX:CX, respectively is shown below. The given program uses ADD and ADC (add with carry) instructions. Run the program and complete the following Table. Also analyze the flags affected.

Address	Data/Code	Mnemonics	Comments
-----: 0100		MOV AX, 1111H	AX = 1111H
-----: 0103		MOV BX, 2323H	BX = 2323H
-----: 0106		MOV CX, 3045H	CX = 3045H
-----: 0109		MOV DX, 4241H	DX = 4241H
-----: 010C		ADD AX, CX	AX = 5366H (1111H + 3045H)
-----: 010F		ADC BX, DX	BX = 6565H (2323H + 4241H + 0, no carry)

Output:

ADD AX, CX



Register Values: AX=4156, BX=2323, CX=3045, DX=4241

IP VALUE:000E

FLAG REGISTER VALUES:

Flags	Output
Overflow Flag	0
Direction Flag	0
Interrupt Flag	1
Sign Flag	0
Zero Flag	0
Auxiliary Carry Flag	0
Parity Flag	1
Carry Flag	0

ADC BX, DX

emulator: noname.bin_

file math debug view external virtual devices virtual drive help

Load

reload

step back

single step

run

step delay ms: 0

registers

H

L

AX

41

56

BX

65

64

CX

30

45

DX

42

41

CS

0100

IP

0010

SS

0100

SP

FFFE

BP

0000

SI

0000

DI

0000

DS

0100

ES

0100

0100:0010

0100:0010

01000: B8 184 3

01001: 11 017 4

01002: 11 017 4

01003: BB 187 5

01004: 23 035 #

01005: 23 035 #

01006: B9 185 6

01007: 45 069 E

01008: 30 048 0

01009: BA 186 7

0100A: 41 065 A

0100B: 42 066 B

0100C: 03 003 8

0100D: C1 193 9

0100E: 13 019 !!

0100F: DA 218 J

01010: 90 144 E

01011: 90 144 E

01012: 90 144 E

01013: 90 144 E

01014: 90 144 E

01015: 90 144 E

MOU AX, 01111h

MOU BX, 02323h

MOU CX, 03045h

MOU DX, 04241h

ADD AX, CX

ADC BX, DX

NOP

NOP

NOP

NOP

NOP

NOP

NOP

NOP

NOP

NOP

NOP

NOP

NOP

...

screen

source

reset

aux

vars

debug

stack

flags

debug log - debug.exe emulation

clear

save to file...

?

AX=4156 BX=6564 CX=3045 DX=4241 SP=FFFE BP=0000 SI=0000 DI=0000

DS=0100 ES=0100 SS=0100 CS=0100 IP=0024 NU UP EI PL NZ NA PO NC

0100:0024 F4 HLT

Register Values: AX=4156, BX=6564, CX=3045, DX=4241

IP VALUE: 0024

Flags	Output
Overflow Flag	0
Direction Flag	0
Interrupt Flag	1
Sign Flag	0
Zero Flag	0
Auxiliary Carry Flag	0
Parity Flag	0
Carry Flag	0

Address	Data/Code	Mnemonics	Comments
-----: 0100		MOV AX, 3000H	AX = 3000H
-----: 0103		MOV BX, 2000H	BX = 2000H
-----: 0106		MOV CX, 1000H	CX = 1000H
-----: 0109		MOV DX, 4000H	DX = 4000H
-----: 010C		SUB AX, CX	AX = AX - CX
-----: 010F		SBB BX, DX	BX = BX - DX - CF (Carry Flag)

[illegible]

```
debug log - debug.exe execution
clear save to file... ?
AX=2000 BX=2000 CX=1000 DX=4000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0100 ES=0100 SS=0100 CS=0100 IP=000E NU UP EI PL NZ NA PE NC
0100:000E 1BD8 SBB BX, DX
```

Register Values: AX=2000 BX=2000 CX=1000 DX=4000
IP VALUE: 000E

FLAG REGISTER VALUES:

Flags	Output
Overflow Flag	0
Direction Flag	0
Interrupt Flag	1
Sign Flag	0
Zero Flag	0
Auxiliary Carry Flag	0
Parity Flag	1
Carry Flag	0

SBB BX, DX

emulator: noname.bin_

file math debug view external virtual devices virtual drive help

Load

reload

step back

single step

run

step delay ms: 0

registers

H

L

AX

20

00

BX

E0

00

CX

10

00

DX

40

00

CS

0100

IP

0024

SS

0100

SP

FFFE

BP

0000

SI

0000

DI

0000

DS

0100

ES

0100

0100:0024

0100:0024

01016: 90 144 E

01017: 90 144 E

01018: 90 144 E

01019: 90 144 E

0101A: 90 144 E

0101B: 90 144 E

0101C: 90 144 E

0101D: 90 144 E

0101E: 90 144 E

0101F: 90 144 E

01020: 90 144 E

01021: 90 144 E

01022: 90 144 E

01023: 90 144 E

01024: F4 244 ↑

01025: 00 000 NULL

01026: 00 000 NULL

01027: 00 000 NULL

01028: 00 000 NULL

01029: 00 000 NULL

0102A: 00 000 NULL

0102B: 00 000 NULL

NOP

NOP

NOP

NOP

NOP

NOP

HLT

ADD [BX + SI], AL

ADD [BX + SI], AL

ADD [BX + SI], AL

ADD [BX + SI], AL

ADD [BX + SI], AL

ADD [BX + SI], AL

ADD [BX + SI], AL

ADD [BX + SI], AL

ADD [BX + SI], AL

ADD [BX + SI], AL

ADD [BX + SI], AL

ADD [BX + SI], AL

ADD [BX + SI], AL

ADD [BX + SI], AL

ADD [BX + SI], AL

...

screen

source

reset

aux

vars

debug

stack

flags

debug log - debug.exe emulation

clear

save to file...

?

AX=2000 BX=E000 CX=1000 DX=4000 SP=FFFE BP=0000 SI=0000 DI=0000

DS=0100 ES=0100 SS=0100 CS=0100 IP=0024 NU UP EI NG NZ NA PE CY

0100:0024 F4 HLT

Register Values: AX=2000 BX=E000 CX=1000 DX=4000
IP VALUE: 0024

FLAG REGISTER VALUES:

Flags	Output
Overflow Flag	0
Direction Flag	0
Interrupt Flag	1
Sign Flag	1
Zero Flag	0
Auxiliary Carry Flag	0
Parity Flag	1
Carry Flag	1

Task 3: Program to multiply two decimal numbers 5 and 10, stored in registers BL and CL respectively is shown below using MUL instruction. The result is stored in register DX. Run the program and complete the following Table. Also analyze the flags affected.

Address	Data/Code	Mnemonics	Comments
-----: 0100		MOV BL, 05H	Load 5 into BL register
-----: 0102		MOV CL, 10H	Load 10 into CL register
-----: 0104		MOV AL, CL	Move the value in CL to AL
-----: 0106		MUL BL	Multiply AL by BL; result in AX
-----: 0108		MOV DX, AX	Move the result in AX to DX

Output:

emulator: noname.bin_

file math debug view external virtual devices virtual drive help

Load reload step back single step run step delay ms: 0

registers H L

AX 00 50 BX 00 05 CX 00 10 DX 00 50 CS 0100 IP 001E SS 0100 SP FFFE BP 0000 SI 0000 DI 0000 DS 0100 ES 0100

0100:001E

01016: 90 144 é
01017: 90 144 é
01018: 90 144 é
01019: 90 144 é
0101A: 90 144 é
0101B: 90 144 é
0101C: 90 144 é
0101D: 90 144 é
0101E: F4 244 ↑
0101F: 00 000 NULL
01020: 00 000 NULL
01021: 00 000 NULL
01022: 00 000 NULL
01023: 00 000 NULL
01024: 00 000 NULL
01025: 00 000 NULL
01026: 00 000 NULL
01027: 00 000 NULL
01028: 00 000 NULL
01029: 00 000 NULL
0102A: 00 000 NULL
0102B: 00 000 NULL

NOP
NOP
NOP
NOP
HLT
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
...

screen source reset aux vars debug stack flags

debug log - debug.exe emulation

clear save to file... ?

AX=0050 BX=0005 CX=0010 DX=0050 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0100 ES=0100 SS=0100 CS=0100 IP=001E NU UP EI PL NZ NA PO NC
0100:001E F4 HLT

Register Values: AX=0050 BX=0005 CX=0010 DX=0050

IP VALUE: 001E

FLAG REGISTER VALUES:

Flags	Output
Overflow Flag	0
Direction Flag	0
Interrupt Flag	1
Sign Flag	0
Zero Flag	0
Auxiliary Carry Flag	0
Parity Flag	0
Carry Flag	0

Task 4: Program to divide a decimal number 10 by a decimal number 5, stored in registers CL and BL respectively is shown below using DIV instruction. The result is stored in register DX. Run the program and complete the following Table. Also analyze the flags affected.

Address	Data/Code	Mnemonics	Comments
-----: 0100		MOV BL, 05H	Initialize BL with 05H
-----: 0102		MOV CL, 10H	Initialize CL with 10H
-----: 0104		MOV AH, 00H	Clear AH (high part of dividend)
-----: 0106		MOV AL, CL	Move CL to AL
-----: 0108		DIV BL	Divide AL (dividend) by BL (divisor)
-----: 010A		MOV DX, AX	Move quotient to DX

Output:

emulator: noname.bin_

file math debug view external virtual devices virtual drive help

Loadreloadstep backsingle steprunstep delay ms: 0

registers

H L

AX0103

BX0005

CX0010

DX0103

CS0100

IP0020

SS0100

SPFFFE

BP0000

SI0000

DI0000

DS0100

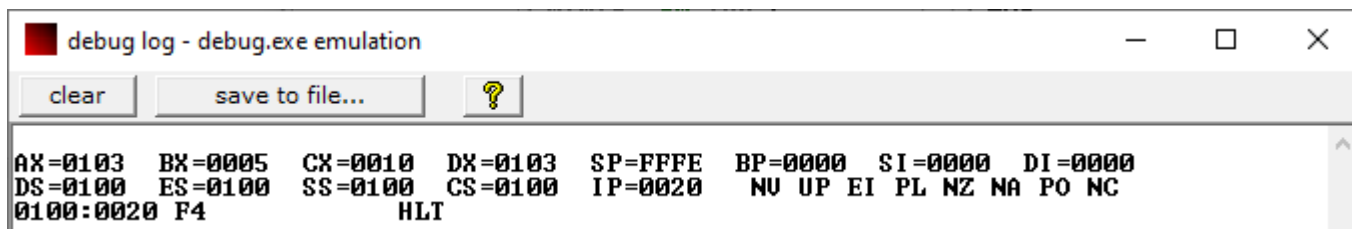
ES0100

0100:0020

01016: 90 144 E
01017: 90 144 E
01018: 90 144 E
01019: 90 144 E
0101A: 90 144 E
0101B: 90 144 E
0101C: 90 144 E
0101D: 90 144 E
0101E: 90 144 E
0101F: 90 144 E
01020: F4 244 ↑
01021: 00 000 NULL
01022: 00 000 NULL
01023: 00 000 NULL
01024: 00 000 NULL
01025: 00 000 NULL
01026: 00 000 NULL
01027: 00 000 NULL
01028: 00 000 NULL
01029: 00 000 NULL
0102A: 00 000 NULL
0102B: 00 000 NULL

0100:0020
NOP
NOP
NOP
NOP
NOP
HLT
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
...

screen source reset aux vars debug stack flags



Register Values: AX=0103 BX=0005 CX=0010 DX=0103

IP VALUE: 0020

FLAG REGISTER VALUES:

Flags	Output
Overflow Flag	0
Direction Flag	0
Interrupt Flag	1
Sign Flag	0
Zero Flag	0
Auxiliary Carry Flag	0
Parity Flag	0
Carry Flag	0

Task 5: Write an Assembly language program to take input number from user and find the square of a number.

Code:

TITLE LAB 7 ACTIVITY 5

.MODEL SMALL

; Define the memory model as small

.STACK 100H

; Set the stack size to 100H

.DATA

MSG DB "ENTER ANY NUMBER: \$"

; Display message prompting the user to enter a number

MSG1 DB 0AH, 0DH, "SQUARE OF \$"

; Display message indicating the purpose of the output

MSG2 DB " IS \$"

; Display message indicating the end of the output

.CODE

MAIN PROC

MOV AX, @DATA

; Initialize the data segment

MOV DS, AX

; Set DS to point to the data segment

AGAIN:

MOV AH, 9

; DOS function to display a string

LEA DX, MSG

INT 21H

; Call DOS interrupt to display the message

MOV AH, 1

; DOS function to read a character from standard input

INT 21H

; Call DOS interrupt

PUSH AX

; Save the entered character

CMP AL, 39H

; Compare if the entered character is greater than '9'

JG AGAIN

; If yes, prompt the user again

CMP AL, 30H	; Compare if the entered character is less than '0'
JL AGAIN	; If yes, prompt the user again
PUSH AX	; Save the valid input character
SUB AL, 30H	; Convert ASCII character to numeric value
MOV BL, AL	; Store the numeric value in BL
MUL BL	; AX = AL * BL
AAM	; Convert the result to two separate digits in AH and AL
MOV BX, AX	; Store the result in BX
MOV AH, 9	; DOS function to display a string
LEA DX, MSG1	
INT 21H	; Call DOS interrupt to display the first part of the output message
POP DX	; Retrieve the original entered character
MOV AH, 2	; DOS function to display a character
INT 21H	; Call DOS interrupt
MOV AH, 9	; DOS function to display a string
LEA DX, MSG2	
INT 21H	; Call DOS interrupt to display the second part of the output message
MOV DL, BH	; Display the high digit of the squared result
ADD DL, 30H	
MOV AH, 02H	; DOS function to display a character
INT 21H	; Call DOS interrupt
MOV DL, BL	; Display the low digit of the squared result
ADD DL, 30H	
MOV AH, 02H	; DOS function to display a character
INT 21H	; Call DOS interrupt
MOV AH, 4CH	; DOS function to exit the program
INT 21H	; Call DOS interrupt
MAIN ENDP	; End of the main procedure
END MAIN	; End of the program with the label MAIN

Output:

 emulator screen (80x25 chars)

```
ENTER ANY NUMBER: 2
SQUARE OF 2 IS 04
```

Lab Exercise #7

Task 5: Write an Assembly language program to take input number from user and find the square of a number.

Code:

```
TITLE LAB 7 ACTIVITY 5                                ; Title of the program

.MODEL SMALL                                           ; Define the memory model as small
.STACK 100H                                           ; Set the stack size to 100H

.DATA
MSG DB 'ENTER A NUMBER (0-9): $'                    ; Define a string message for user input

.CODE
MAIN PROC                                            ; Main procedure starts
    MOV AX, @DATA                                    ; Load the data segment address into AX
    MOV DS, AX                                       ; Set the data segment register to the loaded value

    LEA DX, MSG                                       ; Load effective address of MSG into DX
    MOV AH, 9                                         ; Set the AH register for DOS display function
    INT 21H                                           ; Call DOS interrupt to display the input message

    MOV AH, 1                                         ; Set the AH register for DOS read character function
    INT 21H                                           ; Call DOS interrupt to read a character from the user

    AND AL, 0FH                                       ; Mask the upper bits, ensuring only the lower nibble is considered
    MOV DL, AL                                       ; Move the input digit to DL register
    MUL DL                                           ; Multiply AL by DL, result in AX

MAIN ENDP                                           ; End of main procedure
END MAIN                                             ; End of the program
```

Output:

SCHEM emulator screen (80x25 chars)

```
ENTER A NUMBER (0-9): 3
```

registers		H	L
AX		00	09
BX		00	00
CX		01	36
DX		00	03

flags	
CF	0
ZF	0
SF	0
OF	0
PF	0
AF	0
IF	1
DF	0

Task 6: Write an Assembly language program to take input number from user and find the cube of a number.

Code:

```
TITLE LAB 7 ACTIVITY 6                                ; Title of the program

.MODEL SMALL                                           ; Define the memory model as small
.STACK 100H                                           ; Set the stack size to 100H

.DATA
MSG DB 'ENTER A NUMBER (0-9): $'                     ; Define a string message for user input

.CODE
MAIN PROC                                             ; Main procedure starts
    MOV AX, @DATA                                     ; Load the data segment address into AX
    MOV DS, AX                                       ; Set the data segment register to the loaded value

    LEA DX, MSG                                       ; Load effective address of MSG into DX
    MOV AH, 9                                         ; Set the AH register for DOS display function
    INT 21H                                           ; Call DOS interrupt to display the input message

    MOV AH, 1                                         ; Set the AH register for DOS read character function
    INT 21H                                           ; Call DOS interrupt to read a character from the user

    AND AL, 0FH                                       ; Mask the upper bits, ensuring only the lower nibble is considered
    MOV DL, AL                                       ; Move the input digit to DL register
    MUL DL                                           ; Multiply AL by DL, result in AX
    MUL DL                                           ; Multiply AX by DL, result in AX

MAIN ENDP                                             ; End of main procedure
END MAIN                                             ; End of the program
```

Output:

SC emulator screen (80x25 chars)

```
ENTER A NUMBER <0-9>: 2
```

registers		
	H	L
AX	00	08
BX	00	00
CX	01	38
DX	00	02

flags	
CF	0
ZF	0
SF	0
OF	0
PF	0
AF	0
IF	1
DF	0

Lab Exercise #8

Task 1: Write a program to find the highest number among the following:

68h, 87h, 96h, 45h, 75h

Code:

TITLE LAB 8 ACTIVITY 1

.MODEL SMALL

; Define the memory model as small

.STACK 100H

; Set the stack size to 100H

.DATA

NUMBERS DB 68H, 87H, 96H, 45H, 75H

.CODE

MAIN PROC

MOV AX, @DATA

; Initialize the data segment

MOV DS, AX

; Set DS to point to the data segment

MOV CX, 4

; Set loop counter to the number of elements - 1

MOV BX, OFFSET NUMBERS

; Initialize BX with the offset of NUMBERS

MOV AL, [BX]

; Load the first element into AL

LBACK:

CMP AL, [BX + 1]

; Compare current element with the next one

JC SWAP

; Jump to SWAP if AL is less than [BX + 1]

SBACK:

INC BX

; Move to the next element

LOOP LBACK

; Repeat the loop until CX becomes zero

JMP EXIT

; Jump to EXIT when sorting is done

SWAP:

MOV AL, [BX + 1]

; Swap the elements if needed

JMP SBACK

; Jump to SBACK to compare the swapped element

EXIT:

MOV AH, 4CH

; Exit program

INT 21H

; Call DOS interrupt

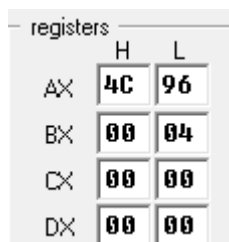
MAIN ENDP

; End of the main procedure

END MAIN

; End of the program with the label MAIN

Output:



Task 2: Write a program to arrange the following numbers in ascending order.

58h, 37h, 96h, 45h, 25h

Code:

TITLE LAB 8 ACTIVITY 2

.MODEL SMALL ; Define the memory model as small

.STACK 100H ; Set the stack size to 100H

.DATA

NUMBER DB 58H, 37H, 96H, 45H, 25H ; Array of numbers to be sorted

.CODE

MAIN PROC

MOV AX, @DATA ; Initialize the data segment

MOV DS, AX ; Set DS to point to the data segment

MOV SI, OFFSET NUMBER ; Initialize SI with the offset of NUMBER

MOV CX, 4 ; Set loop counter to the number of elements - 1

OUTERLOOP:

PUSH CX ; Save the outer loop counter

MOV AL, [SI] ; Load the current element into AL

MOV DI, SI ; Initialize DI with the current offset

INNERLOOP:

INC DI ; Move to the next element

CMP AL, [DI] ; Compare AL with the next element

JBE DONOT_SWAP ; Jump to DONOT_SWAP if AL is less than or equal to [DI]

SWAP:

MOV DL, [DI] ; Store the value of the next element in DL

MOV [DI], AL ; Swap: Store the value of AL in the next element

MOV AL, DL ; Restore AL with the value from DL

DONOT_SWAP:

LOOP INNERLOOP ; Repeat the inner loop until CX becomes zero

MOV [SI], AL ; Store the sorted element back in the array

POP CX ; Restore the outer loop counter

INC SI ; Move to the next element

LOOP OUTERLOOP ; Repeat the outer loop until CX becomes zero

; Display the sorted array

MOV CX, 5 ; Set loop counter to the number of elements

MOV BX, OFFSET NUMBER ; Initialize BX with the offset of NUMBER

DISPLAY:

MOV DL, [BX] ; Load the current element into DL

MOV AH, 2H ; DOS function to display character

INT 21H ; Call DOS interrupt

INC BX ; Move to the next element

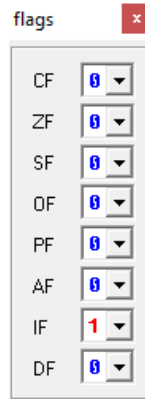
LOOP DISPLAY ; Repeat the display loop until CX becomes zero

MOV AH, 4CH ; Exit program

INT 21H ; Call DOS interrupt

```
; End of the main procedure
; End of the program with the label MAIN
```

Scr emulator screen (80x25 chars)



% = 25h
7 = 37h
E = 45h
X = 58h
û = 96h

Code:

; Title of the program

```
; Define the memory model as small
```

```
; Set the stack size to 100H
```

```
; Define an array of temperature values
```

```
; Main procedure starts
```

```
; Load the data segment address into AX
```

```

; Set the data segment register to the loaded value

```

```
; Initialize SI with the offset of the TEMPERATURE array
```

```
; Initialize CX with the number of elements in the array
```

; Load the first temperature value into AL

```
; Outer loop label
```

; Move to the next element in the array

- **Compare the current temperature value with the next one**

; Jump to SEARCH if the next value is greater or equal

; Update AL with the greater temperature value

; SEARCH label

```

; Repeat the OUTERLOOP until CX becomes zero

```

; Move the final temperature value to DL

```

; Set the AH register for DOS display function

```

```

; Call DOS interrupt to display the result

```

; DOS function to exit the program

```

; Call DOS interrupt to exit

```

```

MAIN ENDP                ; End of main procedure
END MAIN                 ; End of the program

```

Output:



Ascii code

029 = 1D = ➡

registers

	H	L
AX	4C	1D
BX	00	00
CX	00	00
DX	00	1D

flags

CF

1

ZF

0

SF

0

OF

0

PF

1

AF

0

IF

0

DF

0

Task 4: Write a program to convert alphabets of your first name from lowercase to uppercase. The first name should be provided in the form of array.
For example, the first name is "shakir". It should be defined as:

```

.data
first_name db 'shakir'

```

Code:

```

TITLE LAB 8 ACTIVITY 4                ; Title of the program

.MODEL SMALL                          ; Define the memory model as small
.STACK 100H                           ; Set the stack size to 100H

.DATA
FIRST_NAME DB 'faisal'                ; Define a string with the name 'faisal'

.CODE
MAIN PROC                             ; Main procedure starts
    MOV AX, @DATA                     ; Load the data segment address into AX
    MOV DS, AX                        ; Set the data segment register to the loaded value

    MOV SI, OFFSET FIRST_NAME         ; Initialize SI with the offset of the FIRST_NAME string
    MOV CX, 6                         ; Initialize CX with the length of the string

LOOP1:                                ; LOOP1 label
    MOV DL, [SI]                      ; Load the current character into DL
    SUB DL, 20H                       ; Subtract 20H from the ASCII value to convert lowercase to uppercase
    MOV AH, 2                         ; Set the AH register for DOS display function
    INT 21H                           ; Call DOS interrupt to display the uppercase character

```

INC SI	; Move to the next character in the string
LOOP LOOP1	; Repeat LOOP1 until CX becomes zero
MOV AH, 4CH	; DOS function to exit the program
INT 21H	; Call DOS interrupt to exit
MAIN ENDP	; End of main procedure
END MAIN	; End of the program

Output:

 emulator screen (80x25 chars)



FAISAL

Lab Exercise #9

Task 1: Write a program to count the number of 1s in a word entered by user. The count of number is in BCD. Convert the number into decimal number system and display the count on screen.

Code:

```
TITLE LAB 9 ACTIVITY 1                                ; Title of the program

.MODEL SMALL                                           ; Define the memory model as small
.STACK 100H                                           ; Set the stack size to 100H

.DATA
INPUT DB 'ENTER FOUR DIGIT NUMBER $'                ; Message prompt for the user
DIG1 DB ?                                             ; Variable to store the first digit
DIG2 DB ?                                             ; Variable to store the second digit
DIG3 DB ?                                             ; Variable to store the third digit
DIG4 DB ?                                             ; Variable to store the fourth digit

.CODE
MAIN PROC                                             ; Main procedure starts
    MOV AX, @DATA                                     ; Load the address of the data segment into AX
    MOV DS, AX                                       ; Set DS to the address of the data segment

    MOV BX, 0H                                       ; Initialize BX to zero

    MOV DX, OFFSET INPUT                             ; Load the address of the INPUT message into DX
    MOV AH, 9                                        ; Set the AH register for DOS display string function
    INT 21H                                          ; Call DOS interrupt to display the message

    MOV AH, 1                                        ; Set the AH register for DOS character input function
    INT 21H                                          ; Call DOS interrupt to input the first digit
    MOV DIG1, AL                                     ; Store the first digit

    MOV AH, 1                                        ; Set the AH register for DOS character input function
    INT 21H                                          ; Call DOS interrupt to input the second digit
    MOV DIG2, AL                                     ; Store the second digit

    MOV AH, 1                                        ; Set the AH register for DOS character input function
    INT 21H                                          ; Call DOS interrupt to input the third digit
    MOV DIG3, AL                                     ; Store the third digit

    MOV AH, 1                                        ; Set the AH register for DOS character input function
    INT 21H                                          ; Call DOS interrupt to input the fourth digit
    MOV DIG4, AL                                     ; Store the fourth digit

    MOV AL, DIG1                                     ; Move the first digit to AL
    MOV CX, 4                                        ; Set the loop counter to 4

SBACK1:
    ROR AL, 1                                        ; Rotate AL right through the carry flag
    JC COUNT1                                        ; Jump to COUNT1 if the carry flag is set
LBACK1:                                              ; Label LBACK1
    LOOP SBACK1                                     ; Repeat the loop
```

```

    JMP NEXT1                                ; Jump to NEXT1

COUNT1:
    INC BX                                ; Increment BX
    JMP LBACK1                            ; Jump to LBACK1

NEXT1:
    MOV AL, DIG2                          ; Move the second digit to AL
    MOV CX, 4                             ; Set the loop counter to 4

SBACK2:
    ROR AL, 1                             ; Rotate AL right through the carry flag
    JC COUNT2                             ; Jump to COUNT2 if the carry flag is set
LBACK2:                                   ; Label LBACK2
    LOOP SBACK2                           ; Repeat the loop

    JMP NEXT2                              ; Jump to NEXT2

COUNT2:
    INC BX                                ; Increment BX
    JMP LBACK2                            ; Jump to LBACK2

NEXT2:
    MOV AL, DIG3                          ; Move the third digit to AL
    MOV CX, 4                             ; Set the loop counter to 4

SBACK3:
    ROR AL, 1                             ; Rotate AL right through the carry flag
    JC COUNT3                             ; Jump to COUNT3 if the carry flag is set
LBACK3:                                   ; Label LBACK3
    LOOP SBACK3                           ; Repeat the loop

    JMP NEXT3                              ; Jump to NEXT3

COUNT3:
    INC BX                                ; Increment BX
    JMP LBACK3                            ; Jump to LBACK3

NEXT3:
    MOV AL, DIG4                          ; Move the fourth digit to AL
    MOV CX, 4                             ; Set the loop counter to 4

SBACK4:
    ROR AL, 1                             ; Rotate AL right through the carry flag
    JC COUNT4                             ; Jump to COUNT4 if the carry flag is set
LBACK4:                                   ; Label LBACK4
    LOOP SBACK4                           ; Repeat the loop

    JMP GO                                ; Jump to GO

COUNT4:
    INC BX                                ; Increment BX
    JMP LBACK4                            ; Jump to LBACK4

```


GO:

```
MOV AL, BL           ; Move the result (number of set bits) to AL
MOV DL, 01H          ; Move 01H to DL
MUL DL               ; Multiply AL by DL
AAM                  ; Convert AL to ASCII
OR AX, 3030H         ; Convert the result to ASCII
MOV BX, AX           ; Move the ASCII result to BX

MOV AH, 2            ; Set the AH register for DOS display character function
MOV DL, 0DH          ; Load the carriage return character into DL
INT 21H              ; Call DOS interrupt to display the carriage return

MOV DL, 0AH          ; Load the linefeed character into DL
INT 21H              ; Call DOS interrupt to display the linefeed

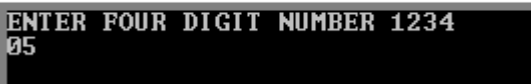
MOV AH, 2            ; Set the AH register for DOS display character function
MOV DL, BH           ; Move the tens place (BH) to DL
INT 21H              ; Call DOS interrupt to display the tens place

MOV AH, 2            ; Set the AH register for DOS display character function
MOV DL, BL           ; Move the ones place (BL) to DL
INT 21H              ; Call DOS interrupt to display the ones place

MOV AH, 4CH          ; Set the AH register for DOS exit function
INT 21H              ; Call DOS interrupt to exit the program

MAIN ENDP            ; End of main procedure
END MAIN              ; End of the program
```

Output:

 emulator screen (80x25 chars)

ENTER FOUR DIGIT NUMBER 1234
05

In BCD codes:

1 = 0001,
2 = 0010,
3 = 0011,
4 = 0100,

Total number of 1s = 05

Task 2: Take two 1-digit inputs (from 0 to 9) from user through keyboard. Write a program to perform the following operations, and display the results for all the operations on screen.

- Addition
- Subtraction
- Multiplication
- Division

- Addition

Code:

```
TITLE LAB 9 ACTIVITY 2A      ; Title of the program
.MODEL SMALL                 ; Define the memory model as small
```

```

.STACK 100H                                ; Set the stack size to 100H

.DATA
FIRST_NUMBER DB 'ENTER FIRST NUMBER: $'      ; Message to prompt user for the first
number
SECOND_NUMBER DB 0DH, 0AH, 'ENTER SECOND NUMBER: $' ; Message to prompt user for the second
number
RESULT DB 0DH, 0AH, 'THE RESULT IS = $'      ; Message to display the result

.CODE                                       ; Beginning of the code segment

MAIN PROC                                ; Start of the main procedure
    MOV AX,@DATA                          ; Move the address of data segment to AX
    MOV DS,AX                             ; Initialize DS register with the data segment address

    LEA DX,FIRST_NUMBER                   ; Load effective address of the first number prompt to DX
    MOV AH,9                             ; DOS function to display a string
    INT 21H                               ; Call DOS interrupt

    MOV AH,1                             ; DOS function to read a character from standard input
    INT 21H                               ; Call DOS interrupt
    MOV BL,AL                             ; Move the input to BL

    LEA DX,SECOND_NUMBER                  ; Load effective address of the second number prompt to DX
    MOV AH,9                             ; DOS function to display a string
    INT 21H                               ; Call DOS interrupt

    MOV AH,1                             ; DOS function to read a character from standard input
    INT 21H                               ; Call DOS interrupt

    MOV AH,00H                           ; Clear AH register
    XCHG AL,BL                           ; Exchange AL and BL
    ADD AL,BL                             ; Add BL to AL
    AAA                                  ; ASCII adjust AL after addition

    OR AX,3030H                           ; Convert result to ASCII in AX
    MOV BX,AX                             ; Move the ASCII value to BX

    LEA DX,RESULT                         ; Load effective address of the result message to DX
    MOV AH,9                             ; DOS function to display a string
    INT 21H                               ; Call DOS interrupt

    MOV AH,2                             ; DOS function to display a character
    MOV DL,BH                             ; Move the high digit of the result to DL
    INT 21H                               ; Call DOS interrupt

    MOV AH,2                             ; DOS function to display a character
    MOV DL,BL                             ; Move the low digit of the result to DL
    INT 21H                               ; Call DOS interrupt

    MOV AH,4CH                           ; DOS function to exit the program
    INT 21H                               ; Call DOS interrupt

```

MAIN ENDP ; End of the main procedure
END MAIN ; End of the program

Output:

 emulator screen (80x25 chars)

```
ENTER FIRST NUMBER: 4
ENTER SECOND NUMBER: 4
THE RESULT IS = 08
```

• Subtraction

Code:

```
TITLE LAB 9 ACTIVITY 2B ; Title of the program
.MODEL SMALL ; Define the memory model as small
.STACK 100H ; Set the stack size to 100H

.DATA
FIRST_NUMBER DB 'ENTER FIRST NUMBER: $' ; Message to prompt user for the first
number
SECOND_NUMBER DB 0DH, 0AH, 'ENTER SECOND NUMBER: $' ; Message to prompt user for the second
number
RESULT DB 0DH, 0AH, 'THE RESULT IS = $' ; Message to display the result

.CODE ; Beginning of the code segment

MAIN PROC ; Start of the main procedure
    MOV AX,@DATA ; Move the address of data segment to AX
    MOV DS,AX ; Initialize DS register with the data segment address

    LEA DX,FIRST_NUMBER ; Load effective address of the first number prompt to DX
    MOV AH,9 ; DOS function to display a string
    INT 21H ; Call DOS interrupt

    MOV AH,1 ; DOS function to read a character from standard input
    INT 21H ; Call DOS interrupt
    MOV BL,AL ; Move the input to BL

    LEA DX,SECOND_NUMBER ; Load effective address of the second number prompt to DX
    MOV AH,9 ; DOS function to display a string
    INT 21H ; Call DOS interrupt

    MOV AH,1 ; DOS function to read a character from standard input
    INT 21H ; Call DOS interrupt

    MOV AH,00H ; Clear AH register
    XCHG AL,BL ; Exchange AL and BL
    SUB AL,BL ; Subtract BL from AL
    AAS ; ASCII adjust AL after subtraction

    OR AX,3030H ; Convert result to ASCII in AX
    MOV BX,AX ; Move the ASCII value to BX
```

LEA DX,RESULT	; Load effective address of the result message to DX
MOV AH,9	; DOS function to display a string
INT 21H	; Call DOS interrupt
MOV AH,2	; DOS function to display a character
MOV DL,BH	; Move the high digit of the result to DL
INT 21H	; Call DOS interrupt
MOV AH,2	; DOS function to display a character
MOV DL,BL	; Move the low digit of the result to DL
INT 21H	; Call DOS interrupt
MOV AH,4CH	; DOS function to exit the program
INT 21H	; Call DOS interrupt

MAIN ENDP	; End of the main procedure
END MAIN	; End of the program

Output:

```

emulator screen (80x25 chars)
ENTER FIRST NUMBER: 4
ENTER SECOND NUMBER: 3
THE RESULT IS = 01
  
```

• Multiplication

Code:

TITLE LAB 9 ACTIVITY 2C	; Title of the program
.MODEL SMALL	; Define the memory model as small
.STACK 100H	; Set the stack size to 100H
.DATA	
FIRST_NUMBER DB 'ENTER FIRST NUMBER: \$'	; Message to prompt user for the first number
SECOND_NUMBER DB 0DH, 0AH, 'ENTER SECOND NUMBER: \$'	; Message to prompt user for the second number
RESULT DB 0DH, 0AH, 'THE RESULT IS = \$'	; Message to display the result
.CODE	; Beginning of the code segment
MAIN PROC	; Start of the main procedure
MOV AX,@DATA	; Move the address of data segment to AX
MOV DS,AX	; Initialize DS register with the data segment address
LEA DX,FIRST_NUMBER	; Load effective address of the first number prompt to DX
MOV AH,9	; DOS function to display a string
INT 21H	; Call DOS interrupt
MOV AH,1	; DOS function to read a character from standard input
INT 21H	; Call DOS interrupt
AND AL,0FH	; Mask high bits to get the numeric value
MOV BL,AL	; Move the input to BL

```

    LEA DX,SECOND_NUMBER           ; Load effective address of the second number prompt to
DX
    MOV AH,9                       ; DOS function to display a string
    INT 21H                       ; Call DOS interrupt

    MOV AH,1                       ; DOS function to read a character from standard input
    INT 21H                       ; Call DOS interrupt

    MOV AH,00H                     ; Clear AH register
    AND AL,0FH                     ; Mask high bits to get the numeric value
    MUL BL                         ; Multiply AX by BL
    AAM                            ; ASCII adjust AX to unpack binary-coded decimal

    OR AX,3030H                    ; Convert result to ASCII in AX
    MOV BX,AX                      ; Move the ASCII value to BX

    LEA DX,RESULT                  ; Load effective address of the result message to DX
    MOV AH,9                       ; DOS function to display a string
    INT 21H                       ; Call DOS interrupt

    MOV AH,2                       ; DOS function to display a character
    MOV DL,BH                      ; Move the high digit of the result to DL
    INT 21H                       ; Call DOS interrupt

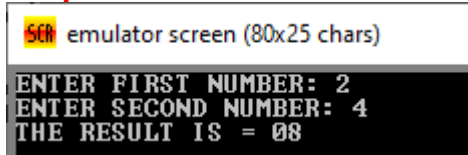
    MOV AH,2                       ; DOS function to display a character
    MOV DL,BL                      ; Move the low digit of the result to DL
    INT 21H                       ; Call DOS interrupt

    MOV AH,4CH                     ; DOS function to exit the program
    INT 21H                       ; Call DOS interrupt

MAIN ENDP                         ; End of the main procedure
END MAIN                          ; End of the program

```

Output:



```

emulator screen (80x25 chars)
ENTER FIRST NUMBER: 2
ENTER SECOND NUMBER: 4
THE RESULT IS = 08

```

• Division

Code:

```

TITLE LAB 9 ACTIVITY 2D           ; Title of the program
.MODEL SMALL                      ; Define the memory model as small
.STACK 100H                       ; Set the stack size to 100H

.DATA
FIRST_NUMBER DB 'ENTER FIRST NUMBER : $' ; Message to prompt user for the first
number
SECOND_NUMBER DB 0DH, 0AH, 'ENTER SECOND NUMBER : $' ; Message to prompt user for the second
number
STR1 DB 0DH, 0AH, 'REMAINDER IS = $' ; Message to display the remainder
STR2 DB 0DH, 0AH, 'QUOTIENT IS = $' ; Message to display the quotient

```

```

.CODE                                     ; Beginning of the code segment

MAIN PROC                               ; Start of the main procedure
    MOV AX,@DATA                         ; Move the address of data segment to AX
    MOV DS,AX                           ; Initialize DS register with the data segment
address

    LEA DX,FIRST_NUMBER                 ; Load effective address of the first number prompt to
DX
    MOV AH,9                            ; DOS function to display a string
    INT 21H                             ; Call DOS interrupt

    MOV AH,1                            ; DOS function to read a character from standard input
    INT 21H                             ; Call DOS interrupt
    AND AL,0FH                          ; Mask high bits to get the numeric value
    MOV BL,AL                           ; Move the input to BL

    LEA DX,SECOND_NUMBER                ; Load effective address of the second number prompt to DX
    MOV AH,9                            ; DOS function to display a string
    INT 21H                             ; Call DOS interrupt

    MOV AH,1                            ; DOS function to read a character from standard input
    INT 21H                             ; Call DOS interrupt

    MOV AH,00H                          ; Clear AH register
    AND AL,0FH                          ; Mask high bits to get the numeric value
    XCHG AL,BL                          ; Exchange AL and BL registers
    AAD                                  ; ASCII adjust AX to unpack binary-coded decimal

    DIV BL                              ; Divide AX by BL; quotient in AL, remainder in AH
    OR AX,3030H                         ; Convert quotient to ASCII in AX
    MOV BX,AX                           ; Move the ASCII value to BX

    LEA DX,STR2                          ; Load effective address of the quotient message to DX
    MOV AH,9                            ; DOS function to display a string
    INT 21H                             ; Call DOS interrupt

    MOV AH,2                            ; DOS function to display a character
    MOV DL,BL                           ; Move the quotient to DL
    INT 21H                             ; Call DOS interrupt

    LEA DX,STR1                          ; Load effective address of the remainder message to DX
    MOV AH,9                            ; DOS function to display a string
    INT 21H                             ; Call DOS interrupt

    MOV AH,2                            ; DOS function to display a character
    MOV DL,BH                           ; Move the remainder to DL
    INT 21H                             ; Call DOS interrupt

    MOV AH,4CH                          ; DOS function to exit the program
    INT 21H                             ; Call DOS interrupt

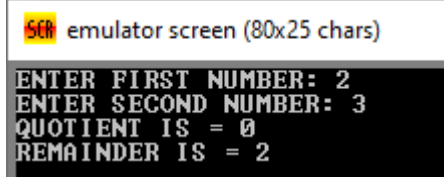
```

```

MAIN ENDP          ; End of the main procedure
END MAIN           ; End of the program

```

Output:



```

emulator screen (80x25 chars)
ENTER FIRST NUMBER: 2
ENTER SECOND NUMBER: 3
QUOTIENT IS = 0
REMAINDER IS = 2

```

Task 3: Write a program to make a counter that counts up from 0 to 99 in BCD and display the result line by line.

Code:

```

TITLE LAB 9 ACTIVITY 3          ; Title of the program

.MODEL SMALL                    ; Define the memory model as small
.STACK 100H                     ; Set the stack size to 100H

.CODE
MAIN PROC                      ; Main procedure starts
    MOV BL, 0                   ; Initialize BL register to 0

AGAIN:
    MOV DL, 1                   ; Load 1 into DL register
    MOV AL, BL                   ; Move the value in BL to AL register

    MUL DL                      ; Multiply AL by DL, result in AX
    AAM                         ; ASCII adjust AX to separate digits

    OR AX, 3030H                ; Convert the digits to ASCII
    MOV CX, AX                  ; Move the ASCII digits to CX register

    MOV AH, 2                   ; Set the AH register for DOS display character function
    MOV DL, CH                   ; Move the high-order digit to DL
    INT 21H                     ; Call DOS interrupt to display the digit

    MOV DL, CL                   ; Move the low-order digit to DL
    INT 21H                     ; Call DOS interrupt to display the digit

    MOV AH, 2                   ; Set the AH register for DOS display character function
    MOV DL, 0AH                 ; Move the linefeed character to DL
    INT 21H                     ; Call DOS interrupt to display the linefeed

    MOV DL, 0DH                 ; Move the carriage return character to DL
    INT 21H                     ; Call DOS interrupt to display the carriage return

    INC BL                      ; Increment BL
    CMP BL, 100                 ; Compare BL with 100
    JNE AGAIN                   ; Jump to AGAIN if not equal

```

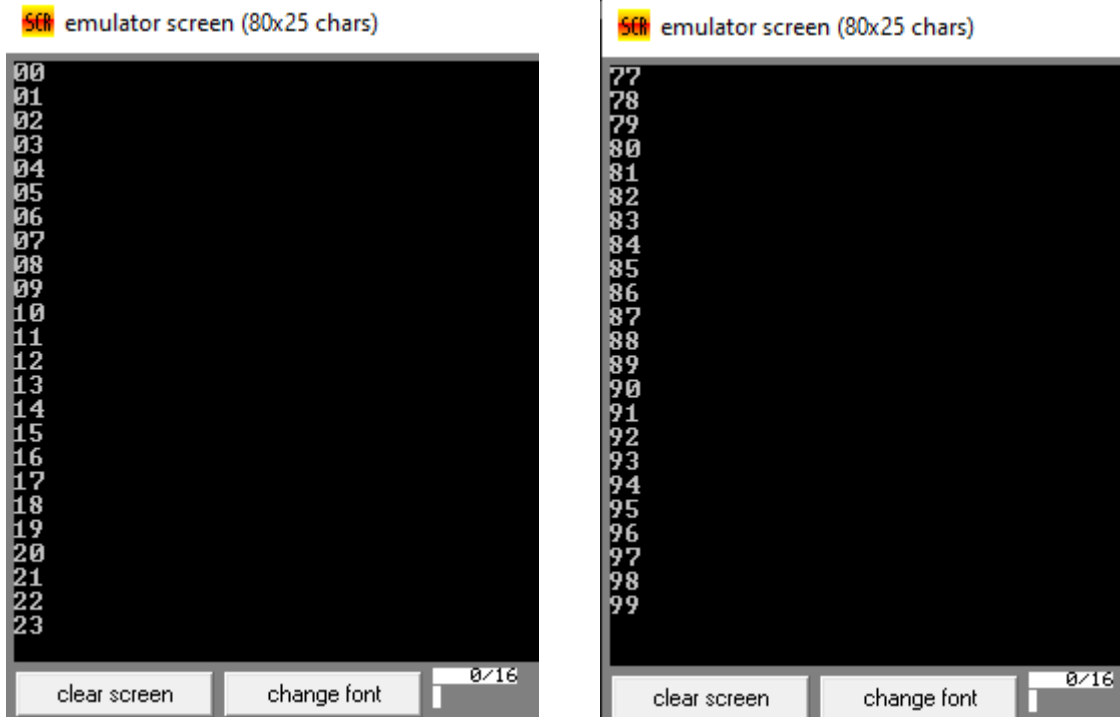
```

MOV AH, 4CH          ; Set the AH register for DOS exit function
INT 21H              ; Call DOS interrupt to exit the program

MAIN ENDP            ; End of main procedure
END MAIN             ; End of the program

```

Output:



Task 4: Write a program to ask user to enter his/her name in lower-case, convert them in upper-case, and display the name on screen. Use buffered input service "0AH" of "INT 21H" to take input from user.

Code:

```

TITLE LAB 9 ACTIVITY 4          ; Title of the program

.MODEL SMALL                    ; Define the memory model as small
.STACK 100H                     ; Set the stack size to 100H

.DATA
FIRSTNAME DB 'ENTER FIRST NAME IN LOWER CASE : $'      ; Message prompt for the user
INPUTNAME DB 10, ?, 10 DUP (?)                          ; Buffer to store user input
UPPERCASE DB 8 DUP ('$')                                ; Buffer to store uppercase version
DISPLAY DB 'YOUR FIRST NAME IS: $'                     ; Message to display the result

.CODE
MAIN PROC                      ; Main procedure starts
    MOV AX, @DATA              ; Load the address of the data segment into AX
    MOV DS, AX                 ; Set DS to the address of the data segment

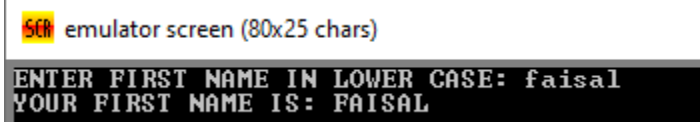
    MOV AH, 9                  ; Set the AH register for DOS display string function
    LEA DX, FIRSTNAME          ; Load the address of the FIRSTNAME message into DX
    INT 21H                    ; Call DOS interrupt to display the message

```


MOV AH, 0AH	; Set the AH register for DOS buffered input function
MOV DX, OFFSET INPUTNAME	; Load the address of INPUTNAME buffer into DX
INT 21H	; Call DOS interrupt to input the name
MOV SI, OFFSET INPUTNAME	; Load the address of INPUTNAME into SI
MOV BX, OFFSET UPPERCASE	; Load the address of UPPERCASE into BX
MOV CX, 7	; Set the loop counter to 7 (assuming 8-character name)
AGAIN:	
MOV AL, [SI] + 2	; Load the next character from INPUTNAME into AL
CMP AL, 61H	; Compare AL with ASCII 'a'
JB OVER	; Jump to OVER if AL is less than 'a'
CMP AL, 7AH	; Compare AL with ASCII 'z'
JA OVER	; Jump to OVER if AL is greater than 'z'
AND AL, 11011111B	; Convert lowercase to uppercase by clearing the 6th bit
OVER:	
MOV [BX], AL	; Store the modified character in UPPERCASE
INC SI	; Increment SI to point to the next character
INC BX	; Increment BX to point to the next position in UPPERCASE
LOOP AGAIN	; Repeat the loop
MOV AH, 2	; Set the AH register for DOS display character function
MOV DL, 0AH	; Load the linefeed character into DL
INT 21H	; Call DOS interrupt to display the linefeed
MOV DL, 0DH	; Load the carriage return character into DL
INT 21H	; Call DOS interrupt to display the carriage return
MOV AH, 9	; Set the AH register for DOS display string function
LEA DX, DISPLAY	; Load the address of the DISPLAY message into DX
INT 21H	; Call DOS interrupt to display the message
MOV AH, 9	; Set the AH register for DOS display string function
MOV DX, OFFSET UPPERCASE	; Load the address of UPPERCASE into DX
INT 21H	; Call DOS interrupt to display the uppercase name
MOV AH, 4CH	; Set the AH register for DOS exit function
INT 21H	; Call DOS interrupt to exit the program
MAIN ENDP	; End of main procedure
END MAIN	; End of the program

Output:

```


emulator screen (80x25 chars)
ENTER FIRST NAME IN LOWER CASE: faisal
YOUR FIRST NAME IS: FAISAL

```

Lab Exercise #10

Task 2: Write three Assembly language procedures to

- **Clear the screen. with the name CLRSCR,**
- **Set the cursor at top left corner of the screen, with the name SETCRSR,**
- **Display the table of any number from 2 to 9, with the name DISPTBL**

Call these procedures in a main program.

Code:

```
TITLE LAB 10 ACTIVITY 2                ; Title of the program
.MODEL SMALL                          ; Memory model for the program
.STACK 100H                           ; Stack size
.DATA
MSG1 DB 'ENTER ANY SINGLE DIGIT NUMBER TO PRINT TABLE: $'    ; Define a string message

DIG DB ?                               ; Declare a variable to store a single-digit number

.CODE
MAIN PROC
    MOV AX,@DATA                      ; Load the data segment address to AX
    MOV DS,AX                         ; Initialize DS register with the data segment address

    CALL CLRSCR                       ; Call procedure to clear the screen
    CALL SETCRSR                      ; Call procedure to set the cursor at the top-left corner
    CALL DISPTBL                      ; Call procedure to display the table

    MOV AH,4CH                       ; DOS function to exit the program
    INT 21H                          ; DOS interrupt to exit
    MAIN ENDP

CLRSCR PROC
    MOV AX,0600H                     ; Function to set video mode
    MOV BH,07                        ; Page number (0 to 7)
    MOV CX,0000                      ; Upper-left corner row
    MOV DX,184FH                     ; Lower-right corner row and column
    INT 10H                          ; Video interrupt to set video mode
    RET                              ; Return from the procedure
CLRSCR ENDP

SETCRSR PROC
    MOV AH,02                        ; Function to set cursor position
    MOV BH,00                        ; Video page number (0 for default)
    MOV DL,00                        ; Column (top-left corner)
    MOV DH,03                        ; Row (top-left corner)
    INT 10H                          ; Video interrupt to set cursor position
    RET                              ; Return from the procedure
SETCRSR ENDP

DISPTBL PROC
    MOV AX,0                         ; Initialize AX to 0
    MOV BX,0                         ; Initialize BX to 0
    MOV CX,0                         ; Initialize CX to 0
```

MOV DX,0	; Initialize DX to 0
MOV AH,9	; Function to print a string
LEA DX,MSG1	; Load the offset address of the message
INT 21H	; DOS interrupt to print the message
MOV AH,1	; Function to read a character
INT 21H	; DOS interrupt to read a character
MOV CX,00001010B	; Set the limit for the table (e.g., 10 times)
MOV DH,1	; Set the initial value for the table
AND AL,0FH	; And AL with 0fh
MOV DIG,AL	; mov al to dig
AGAIN:	
MOV AH,2H	; Function to move the cursor
MOV DL,0AH	; Linefeed character
INT 21H	; DOS interrupt to move the cursor
MOV DL,0DH	; Carriage return character
INT 21H	; DOS interrupt to move the cursor
MOV AL,DIG	; Display the current number
OR AL,30H	; Convert the number to ASCII
MOV AH,2	; Function to display a character
MOV DL,AL	; Move the character to DL
INT 21H	; DOS interrupt to display a character
MOV DL,'x'	; Display the multiplication symbol
INT 21H	; DOS interrupt to display a character
MOV AL,DH	; Display the current iteration
AAM	; ASCII adjust AL to unpack binary-coded decimal
OR AX,3030H	; Convert result to ASCII in AX
MOV BX,AX	; Move the ASCII value to BX
MOV AH,2	; Function to display a character
MOV DL,BH	; Move the high digit of the result to DL
INT 21H	; DOS interrupt to display a character
MOV DL,BL	; Move the low digit of the result to DL
INT 21H	; DOS interrupt to display a character
MOV DL,'='	; Display equal sign
INT 21H	; DOS interrupt to display a character
MOV DL,20H	; Display a space
INT 21H	; DOS interrupt to display a character
MOV AL,DIG	; Calculate and display the result
MUL DH	; Multiply AL by DH
AAM	; ASCII adjust AX to unpack binary-coded decimal
OR AX,3030H	; Convert result to ASCII in AX

```

MOV BX,AX                ; Move the ASCII value to BX

MOV AH,2                 ; Function to display a character
MOV DL,BH                ; Move the high digit of the result to DL
INT 21H                  ; DOS interrupt to display a character

MOV DL,BL                ; Move the low digit of the result to DL
INT 21H                  ; DOS interrupt to display a character

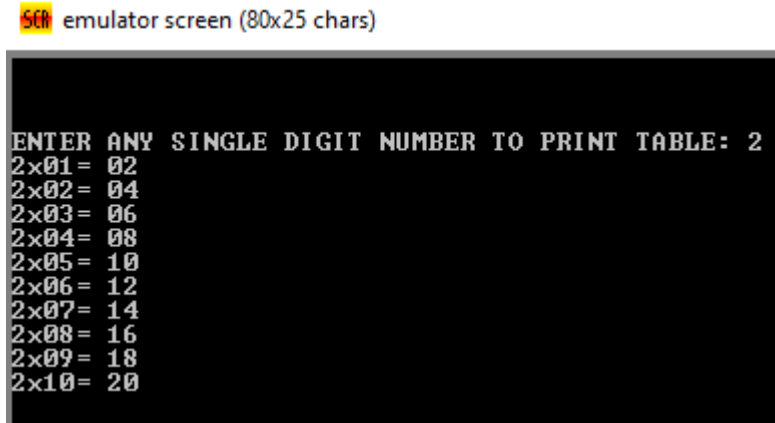
INC DH                   ; Increment DH for the next iteration

LOOP AGAIN               ; Continue the loop until CX reaches the limit
RET                      ; Return from the procedure

DISPTBL ENDP
END MAIN                  ; End of the main program

```

Output:

 emulator screen (80x25 chars)

```

ENTER ANY SINGLE DIGIT NUMBER TO PRINT TABLE: 2
2x01= 02
2x02= 04
2x03= 06
2x04= 08
2x05= 10
2x06= 12
2x07= 14
2x08= 16
2x09= 18
2x10= 20

```

Task 3: Write three Assembly language procedures to

- Take two 1-digit inputs (from 0 to 9) from user, with the name INPNMBR
- Perform the Add, Subtract, Multiply, and Divide operations, name it OPRATIONS,
- Display the results for all the operations on screen, with the name DISPRSLT. Call these procedures in a main program.

Code:

```

TITLE LAB 10 ACTIVITY 3                ; Title of the program
.MODEL SMALL                           ; Memory model
.STACK 100H                            ; Stack size
.DATA
FIRST_NUMBER DB 'ENTER FIRST NUMBER: $' ; Prompt for the first number
SECOND_NUMBER DB 0DH, 0AH, 'ENTER SECOND NUMBER: $' ; Prompt for the second number
ADDITION DB 0DH, 0AH, 'THE RESULT OF ADDITION IS = $' ; Message for addition result
MULTIPLY DB 0DH, 0AH, 'THE RESULT OF MULTIPLICATION IS = $' ; Message for multiplication result
SUBTRACT DB 0DH, 0AH, 'THE RESULT OF SUBTRACTION IS = $' ; Message for subtraction result
REMAINDER DB 0DH, 0AH, 'THE RESULT OF DIVISION, REMAINDER = $' ; Message for division remainder result
QUOTIENT DB 0DH, 0AH, 'THE RESULT OF DIVISION, QUOTIENT = $' ; Message for division quotient result

VAR1 DB ? ; Variable for the first input
VAR2 DB ? ; Variable for the second input
A1 DB ? ; Variables for addition result

```

```

A2 DB ?
M1 DB ?                ; Variables for multiplication result
M2 DB ?
S1 DB ?                ; Variables for subtraction result
S2 DB ?
Q DB ?                 ; Variables for division quotient result
R DB ?                 ; Variable for division remainder result

.CODE
MAIN PROC
    MOV AX,@DATA        ; Load the address of the data segment to AX
    MOV DS,AX           ; Initialize the data segment register with the address in AX

    CALL INPNMBR         ; Call input number procedure
    CALL OPERATIONS      ; Call operations procedure
    CALL DISPRSLT        ; Call display result procedure

    MOV AH,4CH           ; DOS function to terminate program
    INT 21H              ; Call DOS interrupt
MAIN ENDP

INPNMBR PROC
    LEA DX,FIRST_NUMBER ; Load effective address of the first number prompt to DX
    MOV AH,9             ; DOS function to display a string
    INT 21H              ; Call DOS interrupt

    MOV AH,1             ; DOS function to read a character from standard input
    INT 21H              ; Call DOS interrupt
    MOV VAR1,AL          ; Move the input to VAR1

    LEA DX,SECOND_NUMBER ; Load effective address of the second number prompt to DX
    MOV AH,9             ; DOS function to display a string
    INT 21H              ; Call DOS interrupt

    MOV AH,1             ; DOS function to read a character from standard input
    INT 21H              ; Call DOS interrupt
    MOV VAR2,AL          ; Move the input to VAR2

    RET                  ; Return from the procedure
INPNMBR ENDP

OPERATIONS PROC
;ADDITION:
    MOV AX,00            ; Initialize AX to 0
    MOV BX,00            ; Initialize BX to 0

    MOV BL,VAR1          ; Move VAR1 to BL
    MOV AL,VAR2          ; Move VAR2 to AL
    ADD AL,BL            ; Add VAR2 to VAR1
    AAA                  ; ASCII adjust AL after addition
    OR AX,3030H          ; Convert result to ASCII in AX
    MOV BX,AX            ; Move the ASCII value to BX

```

```
MOV A1,BH ; Store high digit of result in A1
MOV A2,BL ; Store low digit of result in A2
```

;SUBTRACTION:

```
MOV AX,00 ; Initialize AX to 0
MOV BX,00 ; Initialize BX to 0

MOV BL,VAR2 ; Move VAR2 to BL
MOV AL,VAR1 ; Move VAR1 to AL
SUB AL,BL ; Subtract VAR2 from VAR1
AAS ; ASCII adjust AL after subtraction
OR AX,3030H ; Convert result to ASCII in AX
MOV BX,AX ; Move the ASCII value to BX
MOV S1,BH ; Store high digit of result in S1
MOV S2,BL ; Store low digit of result in S2
```

;MULTIPLICATION:

```
MOV AX,00 ; Initialize AX to 0
MOV BX,00 ; Initialize BX to 0

MOV BL,VAR1 ; Move VAR1 to BL
MOV AL,VAR2 ; Move VAR2 to AL
MUL BL ; Multiply VAR1 by VAR2
AAM ; ASCII adjust AX to unpack binary-coded decimal
OR AX,3030H ; Convert result to ASCII in AX
MOV BX,AX ; Move the ASCII value to BX
MOV M1,BH ; Store high digit of result in M1
MOV M2,BL ; Store low digit of result in M2
```

;DIVISION:

```
MOV AX,00 ; Initialize AX to 0
MOV BX,00 ; Initialize BX to 0

MOV BL,VAR1 ; Move VAR1 to BL
MOV AL,VAR2 ; Move VAR2 to AL
XCHG AL,BL ; Exchange AL and BL registers
AAD ; ASCII adjust AX to unpack binary-coded decimal
DIV BL ; Divide AX by BL; quotient in AL, remainder in AH
OR AX,3030H ; Convert quotient to ASCII in AX
MOV R,AH ; Move remainder to R
MOV Q,AL ; Move quotient to Q
```

```
RET ; Return from the procedure
OPERATIONS ENDP
```

DISPRSLT PROC

;ADDITION:

```
LEA DX,ADDITION ; Load effective address of the result message to DX
MOV AH,9 ; DOS function to display a string
INT 21H ; Call DOS interrupt
MOV AH,2 ; DOS function to display a character
```

MOV DL,A1	; Move the high digit of the result to DL
INT 21H	; Call DOS interrupt
MOV AH,2	; DOS function to display a character
MOV DL,A2	; Move the low digit of the result to DL
INT 21H	; Call DOS interrupt
;SUBTRACTION:	
LEA DX,SUBTRACT	; Load effective address of the result message to DX
MOV AH,9	; DOS function to display a string
INT 21H	; Call DOS interrupt
MOV AH,2	; DOS function to display a character
MOV DL,S1	; Move the high digit of the result to DL
INT 21H	; Call DOS interrupt
MOV AH,2	; DOS function to display a character
MOV DL,S2	; Move the low digit of the result to DL
INT 21H	; Call DOS interrupt
;MULTIPLICATION:	
LEA DX,MULTIPLY	; Load effective address of the result message to DX
MOV AH,9	; DOS function to display a string
INT 21H	; Call DOS interrupt
MOV AH,2	; DOS function to display a character
MOV DL,M1	; Move the high digit of the result to DL
INT 21H	; Call DOS interrupt
MOV AH,2	; DOS function to display a character
MOV DL,M2	; Move the low digit of the result to DL
INT 21H	; Call DOS interrupt
;DIVISION:	
LEA DX,QUOTIENT	; Load effective address of the result message to DX
MOV AH,9	; DOS function to display a string
INT 21H	; Call DOS interrupt
MOV AH,2	; DOS function to display a character
MOV DL,Q	; Move the high digit of the result to DL
INT 21H	; Call DOS interrupt
LEA DX,REMAINDER	; Load effective address of the result message to DX
MOV AH,9	; DOS function to display a string
INT 21H	; Call DOS interrupt
MOV AH,2	; DOS function to display a character
MOV DL,R	; Move the high digit of the result to DL
INT 21H	; Call DOS interrupt
RET	; Return from the procedure
DISPRSLT ENDP	
END MAIN	; End of the program

Output:

 emulator screen (80x25 chars)

```
ENTER FIRST NUMBER: 3
ENTER SECOND NUMBER: 2
THE RESULT OF ADDITION IS = 05
THE RESULT OF SUBTRACTION IS = 01
THE RESULT OF MULTIPLICATION IS = 06
THE RESULT OF DIVISION, QUOTIENT = 1
THE RESULT OF DIVISION, REMAINDER = 1
```


Lab Exercise #11

Task 1: Write a program using a look-up table and XLAT to retrieve the value of y in the following equation:

$$y = x^2 + 10$$

where, x value ranges from 0 to 9.

Code:

```
TITLE LAB 11 ACTIVITY 3                ; Title directive

.MODEL SMALL                          ; Memory model for the program
.STACK 100H                           ; Stack size

.DATA                                  ; Data segment
VAL db 10,11,14,19,26,35,46,59,74,91  ; Array of values

MSG DB 'ENTER THE VALUE OF X(0-9) TO GET VALUE OF Y=(X^2)+10 : $' ; Prompt message
MSG1 DB 'VALUE OF Y FOR ENTERED VALUE OF X = $' ; Result message

.CODE                                  ; Code segment
MAIN PROC
    MOV AX,@DATA                      ; Load data segment address to AX
    MOV DS,AX                        ; Initialize DS register with the data segment address

    MOV AH,9                          ; DOS function to print a string
    LEA DX,MSG                        ; Load the offset address of the prompt message
    INT 21H                          ; DOS interrupt to print the message

    mov ah,1                          ; DOS function to read a character from standard input
    int 21h                          ; DOS interrupt
    AND AL,0FH                        ; Mask high bits to get the numeric value
    MOV CL,AL                        ; Store the entered value in CL

    MOV AH,2                          ; DOS function to move the cursor
    MOV DL,0AH                       ; Linefeed character
    INT 21H                          ; DOS interrupt

    MOV AH,2                          ; DOS function to move the cursor
    MOV DL,0DH                       ; Carriage return character
    INT 21H                          ; DOS interrupt

    MOV AH,9                          ; DOS function to print a string
    MOV DX,OFFSET MSG1               ; Load the offset address of the result message
    INT 21H                          ; DOS interrupt

    LEA BX,VAL                       ; Load the offset address of the array VAL to BX
    MOV AL,CL                        ; Move the entered value to AL
    XLAT                             ; Translate AL using the contents of the array VAL

    AAM                              ; ASCII adjust AX to unpack binary-coded decimal
    OR AX,3030H                      ; Convert result to ASCII in AX

    MOV BX,AX                        ; Move the ASCII value to BX
```

MOV AH,2	; DOS function to display a character
MOV DL,BH	; Move the high digit of the result to DL
INT 21H	; DOS interrupt
MOV AH,2	; DOS function to display a character
MOV DL,BL	; Move the low digit of the result to DL
INT 21H	; DOS interrupt
MOV AH,4CH	; DOS function to exit the program
INT 21H	; DOS interrupt
MAIN ENDP	
END MAIN	; End of the program
...	

Output:

 emulator screen (80x25 chars)

```
ENTER THE VALUE OF X<0-9> TO GET VALUE OF Y=<X^2>+10: 2
VALUE OF Y FOR ENTERED VALUE OF X = 14
```

Lab Exercise #11

Task 1: Write a program to take buffered input of string and transfer a string in forward direction.

Code:

```
TITLE LAB 11 ACTIVITY 1          ; Title of the program
.MODEL SMALL                    ; Memory model
.STACK 100H                     ; Stack size
.DATA

MSG1 DB 'WHAT IS YOUR NAME? $'  ; Prompt for user's name
DATA1 DB 10,?,10 DUP (?)        ; Buffer to store user input
DATA2 DB 11 DUP ('$')           ; Buffer to store formatted data

.CODE
MAIN PROC

    MOV AX,@DATA                ; Load the address of the data segment to AX
    MOV DS,AX                   ; Initialize the data segment register with the address in AX
    MOV ES,AX                   ; Initialize the extra segment register with the address in AX

    MOV AH,9                    ; DOS function to display a string
    LEA DX,MSG1                  ; Load effective address of the prompt to DX
    INT 21H                     ; Call DOS interrupt

    MOV AH,2                    ; DOS function to display a character
    MOV DL,13                    ; Move ASCII value for carriage return to DL
    INT 21H                     ; Call DOS interrupt
    MOV DL,10                    ; Move ASCII value for line feed to DL
    INT 21H                     ; Call DOS interrupt

    MOV AH,10                   ; DOS function to input a string
    MOV DX,OFFSET DATA1        ; Load effective address of the buffer to DX
    INT 21H                     ; Call DOS interrupt

    CLD                         ; Clear direction flag for forward movement in string operations

    MOV SI,OFFSET DATA1        ; Source index for source string (DATA1)
    MOV DI, OFFSET DATA2       ; Destination index for destination string (DATA2)

    MOV CX,10                   ; Set the count for the number of characters to copy
    REP MOVSB                   ; Repeat Move String Byte operation to copy CX bytes from DS:SI to ES:DI

    MOV AH,2                    ; DOS function to display a character
    MOV DL,13                    ; Move ASCII value for carriage return to DL
    INT 21H                     ; Call DOS interrupt

    MOV AH,9                    ; DOS function to display a string
    LEA DX,DATA2                 ; Load effective address of the formatted data to DX
    INT 21H                     ; Call DOS interrupt

MAIN ENDP
END MAIN
```

Output:

SCR emulator screen (80x25 chars)

```
WHAT IS YOUR NAME?  
FAISAL  
FAISAL
```

Task 2: Write a program to get the string from user. Scan the string and display the count of each letter used in the string.

Code:

TITLE LAB 11 ACTIVITY 2	; Title of the program
.MODEL SMALL	; Memory model
.STACK 100H	; Stack size
.DATA	

```
MSG1 DB 'ENTER A STRING: $' ; Prompt for user's name
COUNT DB 0DH,0AH,'COUNT OF CHARACTERS: $' ; Buffer to store user input
```

```
.CODE
MAIN PROC
```

```
MOV AX,@DATA      ; Load the address of the data segment to AX
MOV DS,AX          ; Initialize the data segment register with the address in AX
```

```
MOV BL,0 ; Initialize BL to 0
```

```
LEA DX,MSG1      ; Load effective address of the prompt to DX
MOV AH,9         ; DOS function to display a string
INT 21H          ; Call DOS interrupt
```

```
L1:      MOV AH,1          ; DOS function to read a character from standard input
        INT 21H          ; Call DOS interrupt
```

CMP AL,13	; Compare AL (input character) with carriage return (Enter key)
JE JUMP	; Jump to JUMP label if AL equals carriage return

```
CMP AL,20H      ; Compare AL with space character
JE L1           ; Jump to L1 label if AL equals space
```

```
INC BL          ; Increment BL to count characters
JMP L1         ; Jump back to L1 label
```

```
JUMP:
    MOV DX,OFFSET COUNT    ; Load effective address of COUNT to DX
    MOV AH,9               ; DOS function to display a string
    INT 21H                ; Call DOS interrupt
```

```
MOV AL,BL ; Move the count of characters to AL
```

```

AAM                ; ASCII adjust AL after multiplication
OR AX,3030H        ; Convert result to ASCII in AX
MOV BX,AX           ; Move the ASCII value to BX

MOV AH,2            ; DOS function to display a character
MOV DL,BH           ; Move the high digit of the result to DL
INT 21H             ; Call DOS interrupt
MOV DL,BL           ; Move the low digit of the result to DL
INT 21H             ; Call DOS interrupt

MOV AH,4CH          ; DOS function to terminate program
INT 21H             ; Call DOS interrupt

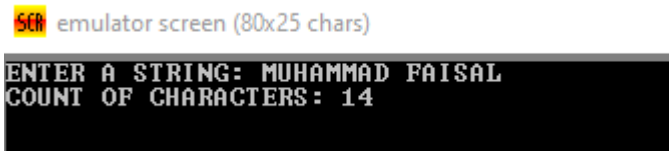
```

```

MAIN ENDP
END MAIN

```

Output:



```

5ch emulator screen (80x25 chars)
ENTER A STRING: MUHAMMAD FAISAL
COUNT OF CHARACTERS: 14

```

Task 3: Write a program to get the string from user as password and store it in a variable called **PASSWORD**. The characters entered by the user must not be displayed on screen and instead of each character, "*" is displayed. Ask user to enter the string again and compare it with the stored password. If the entered string matches the password, display the message "Congratulation! You have entered the correct statement", else display the message "Alas! You have failed to enter the correct statement".

```

Code:
TITLE LAB 11 ACTIVITY 3          ; Title of the program
.MODEL SMALL                     ; Memory model
.STACK 100H                      ; Stack size
.DATA

MSG1 DB 'ENTER THE PASSWORD: $' ; Prompt for the password
MSG2 DB 0DH,0AH,'ENTER IT AGAIN: $' ; Prompt to enter the password again
MSG3 DB 0DH,0AH,'CONGRATULATION! YOU HAVE ENTERED THE CORRECT STATEMENT$'
MSG4 DB 0DH,0AH,'ALAS! YOU HAVE FAILED TO ENTER THE CORRECT STATEMENT$'

PASSWORD DB 50 DUP('$')          ; Buffer to store the original password
PASSWORD_LEN DB ?                ; Variable to store the length of the original password

CMP_PASSWORD DB 50 DUP('$')      ; Buffer to store the password entered for comparison
CMP_PASS_LEN DB ?                ; Variable to store the length of the entered password

.CODE
MAIN PROC

    MOV AX,@DATA                 ; Load the address of the data segment to AX
    MOV DS,AX                   ; Initialize the data segment register with the address in AX

```

```

MOV ES,AX                ; Initialize the extra segment register with the address in AX

CLD                      ; Clear direction flag for forward movement

LEA DX,MSG1              ; Load effective address of the prompt to DX
MOV AH,9                 ; DOS function to display a string
INT 21H                 ; Call DOS interrupt

MOV CX,2                ; Loop twice to get password twice
LEA SI,PASSWORD          ; Load effective address of PASSWORD to SI
LEA DI,PASSWORD_LEN      ; Load effective address of PASSWORD_LEN to DI

INPUT:
MOV AH,7                ; DOS function to read a character from standard input
INT 21H                 ; Call DOS interrupt

CMP AL,13               ; Compare AL (input character) with carriage return (Enter key)
JE NEXT1                ; Jump to NEXT1 if Enter key is pressed

MOV [SI],AL             ; Move the input character to PASSWORD buffer
INC SI                  ; Move to the next position in PASSWORD buffer

ADD [DI],1              ; Increment the length of the PASSWORD

MOV AH,2                ; DOS function to display a character
MOV DL,*                ; Display '*' instead of the entered character
INT 21H                 ; Call DOS interrupt

JMP INPUT               ; Jump back to INPUT to get the next character

NEXT1:
DEC CX                  ; Decrement CX to control the loop
JZ NEXT2                ; Jump to NEXT2 if both passwords are obtained

MOV DX,OFFSET MSG2      ; Load effective address of MSG2 to DX
MOV AH,9                 ; DOS function to display a string
INT 21H                 ; Call DOS interrupt

LEA SI,CMP_PASSWORD     ; Load effective address of CMP_PASSWORD to SI
LEA DI,CMP_PASS_LEN     ; Load effective address of CMP_PASS_LEN to DI

JMP INPUT               ; Jump back to INPUT to get the next character

NEXT2:
CALL COMPARE            ; Call the COMPARE procedure

MOV AH,4CH              ; DOS function to terminate program
INT 21H                 ; Call DOS interrupt

MAIN ENDP

```

```

COMPARE PROC
    MOV AL,PASSWORD_LEN          ; Move the length of PASSWORD to AL
    CMP AL,CMP_PASS_LEN          ; Compare with the length of CMP_PASSWORD
    JNZ FALSE                     ; Jump to FALSE if lengths are not equal

    MOV CL,PASSWORD_LEN          ; Move the length to CL for string comparison

    MOV SI,OFFSET PASSWORD        ; Load effective address of PASSWORD to SI
    MOV DI,OFFSET CMP_PASSWORD    ; Load effective address of CMP_PASSWORD to DI

    REPE CMPSB                    ; Repeat the string compare operation

    JNZ FALSE                     ; Jump to FALSE if strings are not equal
    JMP TRUE                      ; Jump to TRUE if strings are equal

FALSE:
    MOV DX,OFFSET MSG4           ; Load effective address of MSG4 to DX
    MOV AH,9                     ; DOS function to display a string
    INT 21H                      ; Call DOS interrupt

    JMP EXIT                     ; Jump to EXIT

TRUE:
    MOV DX,OFFSET MSG3           ; Load effective address of MSG3 to DX
    MOV AH,9                     ; DOS function to display a string
    INT 21H                      ; Call DOS interrupt

    JMP EXIT                     ; Jump to EXIT

EXIT:
    RET                          ; Return from the procedure

COMPARE ENDP
END MAIN

```

Output:

 emulator screen (80x25 chars)

```

ENTER THE PASSWORD: *****
ENTER IT AGAIN: *****
CONGRATULATION! YOU HAVE ENTERED THE CORRECT STATEMENT

```

 emulator screen (80x25 chars)

```

ENTER THE PASSWORD: *****
ENTER IT AGAIN: ***
ALAS! YOU HAVE FAILED TO ENTER THE CORRECT STATEMENT

```

 emulator screen (80x25 chars)

```

ENTER THE PASSWORD: *****
ENTER IT AGAIN: *****
ALAS! YOU HAVE FAILED TO ENTER THE CORRECT STATEMENT

```

Lab Exercise #12

Task 1: Write a program that takes a single character input and displays it in a new line.

Code:

```
OUTPUT MACRO STRING                ; Define a macro to display a string
    MOV AH,9                       ; DOS function to display a string
    MOV DX,OFFSET STRING           ; Load effective address of STRING to DX
    INT 21H                        ; Call DOS interrupt
ENDM

INPUT MACRO F_NO                   ; Define a macro for input with function number
    MOV AH,F_NO                    ; Move the function number to AH
    INT 21H                        ; Call DOS interrupt
    MOV CL,AL                      ; Move the input character to CL
ENDM

CRLF MACRO CR,LF                   ; Define a macro for carriage return and line feed
    MOV AH,2                       ; DOS function to display a character
    MOV DL,CR                      ; Move ASCII value for carriage return to DL
    INT 21H                        ; Call DOS interrupt
    MOV AH,2                       ; DOS function to display a character
    MOV DL,LF                      ; Move ASCII value for line feed to DL
    INT 21H                        ; Call DOS interrupt
ENDM

DISPLAY MACRO F_NO                 ; Define a macro for displaying with function number
    MOV AH,F_NO                    ; Move the function number to AH
    MOV DL, CL                     ; Move the value in CL to DL
    INT 21H                        ; Call DOS interrupt
ENDM

TITLE LAB 12 ACTIVITY 1            ; Title of the program
.MODEL SMALL                       ; Memory model
.STACK 100H                        ; Stack size
.DATA

MSG1 DB 'ENTER A NUMBER: $'        ; Prompt to enter a number

.CODE

MAIN PROC

    MOV AX,@DATA                   ; Load the address of the data segment to AX
    MOV DS,AX                      ; Initialize the data segment register with the address in AX

    OUTPUT MSG1                    ; Use OUTPUT macro to display MSG1

    INPUT 1                        ; Use INPUT macro with function number 1

    CRLF 13,10                     ; Use CRLF macro to display carriage return and line feed
```

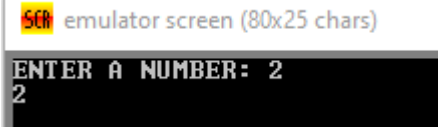

DISPLAY 2

; Use DISPLAY macro with function number 2

MAIN ENDP

END MAIN

Output:

The screenshot shows a window titled 'scm emulator screen (80x25 chars)'. Inside the window, the text 'ENTER A NUMBER: 2' is displayed on the first line, and the character '2' is on the second line, indicating user input.

Task 2: Write a program to display your bio data using carriage return and line feed macro.

Code:

```
OUTPUT MACRO STRING                ; Define a macro to display a string
    MOV AH,9                        ; DOS function to display a string
    MOV DX,OFFSET STRING            ; Load effective address of STRING to DX
    INT 21H                          ; Call DOS interrupt
ENDM

CRLF MACRO CR,LF                    ; Define a macro for carriage return and line feed
    MOV AH,2                        ; DOS function to display a character
    MOV DL,CR                       ; Move ASCII value for carriage return to DL
    INT 21H                          ; Call DOS interrupt
    MOV AH,2                        ; DOS function to display a character
    MOV DL,LF                       ; Move ASCII value for line feed to DL
    INT 21H                          ; Call DOS interrupt
ENDM

TITLE LAB 12 ACTIVITY 1              ; Title of the program
.MODEL SMALL                         ; Memory model
.STACK 100H                          ; Stack size
.DATA

T1 DB 'BIO DATA$'                  ; Title string
D1 DB "NAME: FAISAL$"               ; Data string 1
D2 DB "AGE: 20$"                    ; Data string 2
D3 DB "GENDER: MALE$"              ; Data string 3

.CODE

MAIN PROC

    MOV AX,@DATA                    ; Load the address of the data segment to AX
    MOV DS,AX                       ; Initialize the data segment register with the address in AX

    OUTPUT T1                        ; Use OUTPUT macro to display title

    CRLF 13,10                      ; Use CRLF macro to display carriage return and line feed

    CRLF 13,10                      ; Use CRLF macro to create space
```

OUTPUT D1 ; Use OUTPUT macro to display data 1

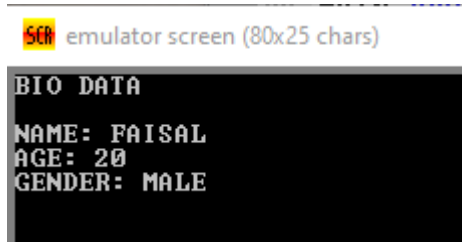
CRLF 13,10 ; Use CRLF macro to create space
OUTPUT D2 ; Use OUTPUT macro to display data 2

CRLF 13,10 ; Use CRLF macro to create space
OUTPUT D3 ; Use OUTPUT macro to display data 3

MAIN ENDP

END MAIN

Output:



The screenshot shows a terminal window titled "Scr emulator screen (80x25 chars)". The output displayed is:

```
BIO DATA
NAME: FAISAL
AGE: 20
GENDER: MALE
```

Task 3: Write a macro that multiplies two words by repeated addition, and then saves the result. Write a program that multiplies the following numbers using macro:

- 2000 x 500
- 2500 x 500
- 300 x 400

Code:

```
TITLE LAB 12 ACTIVITY 3                ; SET THE TITLE OF THE PROGRAM

REP_ADDITION MACRO N01,N02             ; DEFINE A MACRO NAMED REP_ADDITION
    LOCAL LOOP1                        ; DECLARE A LOCAL LABEL 'LOOP1'
    MOV CX, N02                        ; MOVE THE VALUE OF N02 INTO THE CX REGISTER
    MOV AX, N01                        ; MOVE THE VALUE OF N01 INTO THE AX REGISTER
    LOOP1:                             ; START OF THE LOOP LABEL 'LOOP1'
    ADD AX, N01                        ; ADD THE VALUE OF N01 TO AX
    LOOP LOOP1                         ; LOOP TO THE LABEL SPECIFIED BY LOOP1 WHILE CX IS NOT ZERO
ENDM

.MODEL SMALL                           ; DEFINE THE MEMORY MODEL AS SMALL
.STACK 100H                           ; SET THE STACK SIZE TO 100H
.CODE

MAIN PROC                             ; START OF THE MAIN PROCEDURE
    REP_ADDITION 2000,500              ; CALL THE REP_ADDITION MACRO WITH PARAMETERS
    MOV DX, AX                         ; MOVE THE VALUE OF AX INTO DX
    REP_ADDITION 2500,500              ; CALL THE REP_ADDITION MACRO WITH PARAMETERS
    MOV BX, AX                         ; MOVE THE VALUE OF AX INTO BX
    REP_ADDITION 300,400               ; CALL THE REP_ADDITION MACRO WITH PARAMETERS
ENDP                                   ; END OF THE MAIN PROCEDURE
END MAIN                              ; END OF THE PROGRAM
```

Output:

registers	
	H L
AX	D5 EC
BX	1C 94
CX	00 00
DX	4A 10

Task 4: Write a macro to calculate total wages which is equal to the sum of overtime, salary and bonuses. Write a program by using following values:

- Salary = 60000, Overtime = 2500 and Bonuses = 3000
- Salary = 45000, Overtime = 1000 and Bonuses = 500
- Salary=10000, Overtime = 500 and Bonuses = 100

Code:

```
ADDITION MACRO BASIC, OVERTIME, BONUS  ; DEFINE A MACRO NAMED ADDITION
    MOV AX, BASIC                       ; MOVE THE VALUE OF BASIC INTO THE AX REGISTER
    MOV BX, OVERTIME                   ; MOVE THE VALUE OF OVERTIME INTO THE BX REGISTER
    MOV CX, BONUS                      ; MOVE THE VALUE OF BONUS INTO THE CX REGISTER
    ADD AX, BX                         ; ADD THE VALUE OF BX TO AX
```

```

    ADD AX, CX                ; ADD THE VALUE OF CX TO AX
ENDM

TITLE LAB 12 ACTIVITY 4      ; SET THE TITLE OF THE PROGRAM
.MODEL SMALL                 ; DEFINE THE MEMORY MODEL AS SMALL
.STACK 100H                  ; SET THE STACK SIZE TO 100H

.CODE

MAIN PROC                    ; START OF THE MAIN PROCEDURE

    ADDITION 60000,2500,3000 ; CALL THE ADDITION MACRO WITH PARAMETERS
    MOV DX,AX                ; MOVE THE VALUE OF AX INTO DX

    ADDITION 45000,1000,500  ; CALL THE ADDITION MACRO WITH PARAMETERS
    MOV BX,AX                ; MOVE THE VALUE OF AX INTO BX

    ADDITION 1000,500,100    ; CALL THE ADDITION MACRO WITH PARAMETERS

    MOV AH, 4CH              ; SET THE DOS FUNCTION TO EXIT THE PROGRAM
    INT 21H                  ; CALL DOS INTERRUPT

MAIN ENDP                    ; END OF THE MAIN PROCEDURE
END MAIN                     ; END OF THE PROGRAM

```

Output:

registers		H	L
AX		4C	40
BX		01	F4
CX		00	64
DX		FF	DC

Lab Exercise #13

Task 1: Implement the above given code in emu8086 with the settings given. Make your own operating system with the help of given code.

Code:

```
name "loader"
; this is a very basic example of a tiny operating system.

; directive to create boot file:
#make_boot#

; this is an os loader only!
;
; it can be loaded at the first sector of a floppy disk:

; cylinder: 0
; sector: 1
; head: 0

;=====
; how to test micro-operating system:
; 1. compile micro-os_loader.asm
; 2. compile micro-os_kernel.asm
; 3. compile writebin.asm
; 4. insert empty floppy disk to drive a:
; 5. from command prompt type:
;     writebin loader.bin
;     writebin kernel.bin /k
;=====

;
;
; The code in this file is supposed to load
; the kernel (micro-os_kernel.asm) and to pass control over it.
; The kernel code should be on floppy at:

; cylinder: 0
; sector: 2
; head: 0

; memory table (hex):
;-----
; 07c0:0000 | boot sector
; 07c0:01ff | (512 bytes)
;-----
; 07c0:0200 | stack
; 07c0:03ff | (255 words)
;-----
; 0800:0000 | kernel
; 0800:1400 |
;          | (currently 5 kb,
;          | 10 sectors are
;          | loaded from
```

```
; | floppy)
; -----
```

```
; To test this program in real environment write it to floppy
; disk using compiled writebin.asm
; After successfully compilation of both files,
; type this from command prompt: writebin loader.bin
```

```
; Note: floppy disk boot record will be overwritten.
; the floppy will not be useable under windows/dos until
; you reformat it, data on floppy disk may be lost.
; use empty floppy disks only.
```

```
; micro-os_loader.asm file produced by this code should be less or
; equal to 512 bytes, since this is the size of the boot sector.
```

```
; boot record is loaded at 0000:7c00
org 7c00h
```

```
; initialize the stack:
mov ax, 07c0h
mov ss, ax
mov sp, 03feh ; top of the stack.
```

```
; set data segment:
xor ax, ax
mov ds, ax
```

```
; set default video mode 80x25:
mov ah, 00h
mov al, 03h
int 10h
```

```
; print welcome message:
lea si, msg
call print_string
```

```
;=====
; load the kernel at 0800h:0000h
; 10 sectors starting at:
; cylinder: 0
; sector: 2
; head: 0
```

```
; BIOS passes drive number in dl,
; so it's not changed:
```

```
mov ah, 02h ; read function.
mov al, 10 ; sectors to read.
mov ch, 0 ; cylinder.
mov cl, 2 ; sector.
```

```

mov    dh, 0 ; head.
; dl not changed! - drive number.

; es:bx points to receiving
; data buffer:
mov    bx, 0800h
mov    es, bx
mov    bx, 0

; read!
int    13h
;=====

; integrity check:
cmp    es:[0000], 0E9h ; first byte of kernel must be 0E9 (jmp).
je     integrity_check_ok

; integrity check error
lea    si, err
call   print_string

; wait for any key...
mov    ah, 0
int    16h

; store magic value at 0040h:0072h:
; 0000h - cold boot.
; 1234h - warm boot.
mov    ax, 0040h
mov    ds, ax
mov    w.[0072h], 0000h ; cold boot.
jmp    0ffffh:0000h ; reboot!

;=====

integrity_check_ok:
; pass control to kernel:
jmp    0800h:0000h

;=====

print_string proc near
push    ax ; store registers...
push    si ;
next_char:
    mov    al, [si]
    cmp    al, 0
    jz     printed
    inc    si
    mov    ah, 0eh ; teletype function.
    int    10h
    jmp    next_char
printed:
pop     si ; re-store registers...

```

```
pop    ax    ;
ret
print_string endp
```

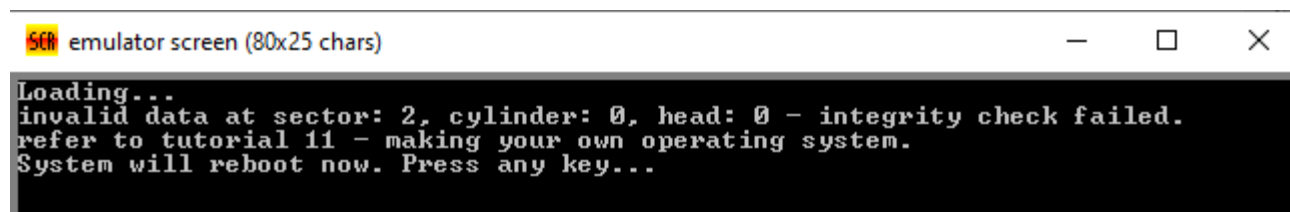
```
;==== data section =====
```

```
msg db "Loading...",0Dh,0Ah,0
```

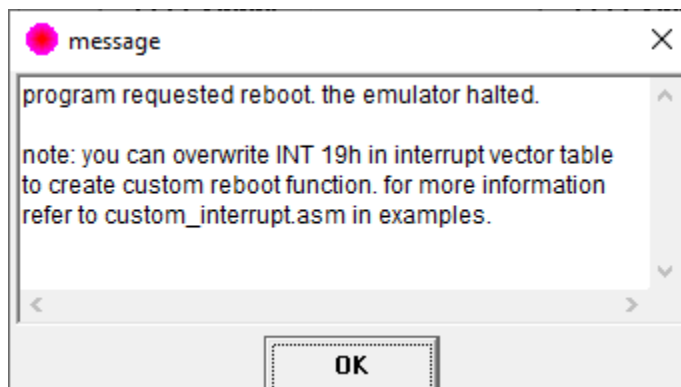
```
err db "invalid data at sector: 2, cylinder: 0, head: 0 - integrity check failed.", 0Dh,0Ah
     db "refer to tutorial 11 - making your own operating system.", 0Dh,0Ah
     db "System will reboot now. Press any key...", 0
```

```
;=====
```

Output:



```
emulator screen (80x25 chars)
Loading...
invalid data at sector: 2, cylinder: 0, head: 0 - integrity check failed.
refer to tutorial 11 - making your own operating system.
System will reboot now. Press any key...
```



Lab Exercise #14

Task 1: Open Ended Lab could be based on implementation of Assembly language programming.

Implement a computer system scenario of your choice. Complete a task which is different from those taught in class/laboratory or a similar system with enhancements/improvements clearly demonstrated. You can select any specific technique or a system from a research paper. The details should be well documented. Codes should be with comments.

Topics may be include (but not limited to) the concepts discussed in previous labs related to BIOS and DOS interrupt services, program flow control, and Arithmetic operations.

Code:

```
.MODEL SMALL                ; Define the memory model as small
.STACK 100H                 ; Set the stack size to 100H
.DATA

INPUT DB "INPUT A NUMBER: $" ; Prompt for user input
EVEN DB 0AH,0DH, "NUMBER IS EVEN$" ; Message for even number
ODD DB 0AH,0DH, "NUMBER IS ODD$" ; Message for odd number
DATA1 DB 10 DUP(?)          ; Array to store user input

.CODE
MAIN PROC
    MOV AX,@DATA             ; Initialize AX with the address of the data segment
    MOV DS,AX               ; Set DS to point to the data segment

    MOV AH,9                ; DOS function to print string
    LEA DX,INPUT             ; Load address of INPUT into DX
    INT 21H                 ; Call DOS interrupt to print the prompt

    MOV SI,OFFSET DATA1     ; Initialize source index for input

INPUT1:
    MOV AH,1                ; DOS function to read a character from standard input
    INT 21H                 ; Call DOS interrupt

    CMP AL,0DH              ; Check if Enter key is pressed
    JE CHECK                ; If Enter is pressed, jump to CHECK

    MOV [SI],AL              ; Store the entered digit in DATA1 array
    INC SI                  ; Move to the next index

    JMP INPUT1               ; Repeat input until Enter is pressed

CHECK:
    MOV AL,[SI-1]           ; Load the last entered character
    AND AL,0FH              ; Mask upper bits to get the actual digit
    MOV BL,2                ; Set BL to 2 for division
    DIV BL                  ; Divide the digit by 2

    CMP AH,0                ; Check if remainder is 0 (even)
    JE EVEN1                ; If even, jump to EVEN1
    JMP ODD1                ; If odd, jump to ODD1
```

EVEN1:

```
MOV AH,9           ; DOS function to print string
LEA DX,EVEN        ; Load address of EVEN into DX
INT 21H            ; Call DOS interrupt to print the even message
JMP EXIT           ; Jump to EXIT
```

```
ODD1:
MOV AH,9           ; DOS function to print string
LEA DX,ODD         ; Load address of ODD into DX
INT 21H            ; Call DOS interrupt to print the odd message
```

```
EXIT:
MOV AH,4CH         ; DOS function to terminate program
INT 21H            ; Call DOS interrupt
```

```
MAIN ENDP
END MAIN
```

Output:

 emulator screen (80x25 chars)

```
INPUT A NUMBER: 305
NUMBER IS ODD
```

 emulator screen (80x25 chars)

```
INPUT A NUMBER: 278
NUMBER IS EVEN
```